

PART 3 – MORE 3D MODELING + ANALYSIS

The third part of the primer is the final section that deals exclusively with the basic modeling in *FlexSim*'s 3D environment. Of course, as has been stressed throughout the primer, this is only an introduction to the capabilities of *FlexSim* simulation software. Therefore, there are many more features and capabilities available in the 3D environment for effectively building and analyzing simulation models.

This part also introduces the basics of using simulation models for analysis.

In particular, the sections in this part of the primer describe:

- Combining and separating flowitems through an example that is an extension of the finishing area model from the previous sections. The extension is to a packing area where the finished containers are packed with a variety of components.
- An alternative means to control the travel path of Task Executors, in this case the finish operators, by using the A* algorithm.
- Using Lists as a means for representing complex flow rules. In this case, how the finish operator decides which container to process next.
- Setting up *FlexSim*'s Experimenter to compare multiple options based on key performance measures

Subsequent parts of the primer describe experimentation and analysis with *FlexSim* and building complex operations logic, both using ProcessFlow and coding in FlexScript.

1 COMBINING AND SEPARATING FLOWITEMS

The model from the previous section is extended to include a packing area. In this area, containers are packed with components that are produced elsewhere. At this time, it is assumed that the components arrive at the packing area on a schedule and in batches. In a subsequent section, the model is modified so that the components arrive based on current inventory levels and specified reorder points.

Also, in this example, there are only two types of components – referred to as A and B, which are purple boxed-shaped items and white cylinder-shaped items, respectively. Of course, the model could consider any number of components, but two is sufficient to introduce some key concepts. The model is developed in such a way that scaling up to a model including more components is quite easy.



As has been mentioned before, but it is a very important concept, it is good modeling practice to build smaller models first in order to test, validate, verify the approach and methods, then scale up to represent the real system.

Arriving batches wait in a buffer until the finish operator travels to the batch. The operator then unloads the individual components in the batch so that they can be used in the packing operation. Each container type is packed with its own mix of components. However, initially, each container will contain the same component mix.

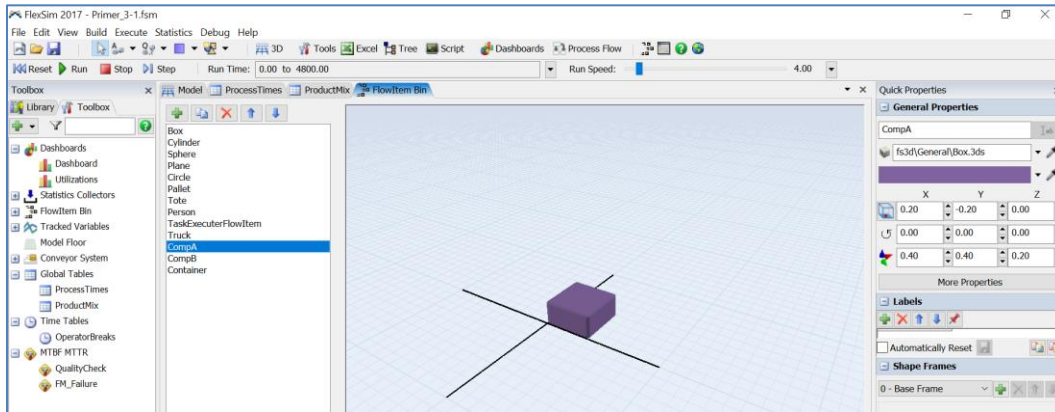


Begin with the basic model from the previous section, named Primer_2-5 and **Save As** Primer_3-1. Basically make a copy of the model so that it can be customized and the original model is retained.

1.1 Scheduled arrivals of components

The first step is to create two new flowitems to represent components A and B. This is done via the FlowItem Bin, which is accessed by the button on the Main Menu bar (to the right of the Process Flow button) or through the Toolbox.

- Create the first component as follows and as showed in the figure below.
 - Duplicate the **Box** item by highlighting the box in the list of flowitems (Box, Cylinder, Sphere, ...) and pressing the **Duplicate the selected flowitem** button (to the right of the + button). The result is a new flowitem at the bottom of the list called “Box copy.”
 - In the Quick Properties interface:
 - Change the name of the flowitem from Box copy to **CompA**.
 - Change its color from brown to purple using the color palette.
 - Change its size from the default values (x=0.61, y=0.61, z=0.3) to x=0.4, y=0.4, z=0.2.



- Repeat the above process for the second component.
 - Duplicate the **Cylinder** item: highlight the cylinder in the list of flowitems (Box, Cylinder, Sphere, ...) and press the **Duplicate the selected flowitem** button (to the right of the + button). The result is a new flowitem at the bottom of the list called "Cylinder copy."
 - In the Quick Properties interface:
 - Change the name of the flowitem from Cylinder copy to **CompB**.
 - Change its color from brown to white using the color palette.
 - Change its size from the default values ($x=0.70$, $y=0.70$, $z=0.8$) to $x=0.2$, $y=0.2$, $z=0.2$.
- Repeat the above process for the container.
 - Duplicate the **Tote** item: highlight the tote in the list of flowitems (Box, Cylinder, Sphere, ...) and press the **Duplicate the selected flowitem** button (to the right of the + button). The result is a new flowitem at the bottom of the list called "Tote copy."
 - In the Quick Properties interface:
 - Change the name of the flowitem from Tote copy to Container.
 - Change its size from the default values ($x=1.0$, $y=0.70$, $z=0.55$) to $x=0.50$, $y=0.50$, $z=0.50$.
 - Leave its color as the default.

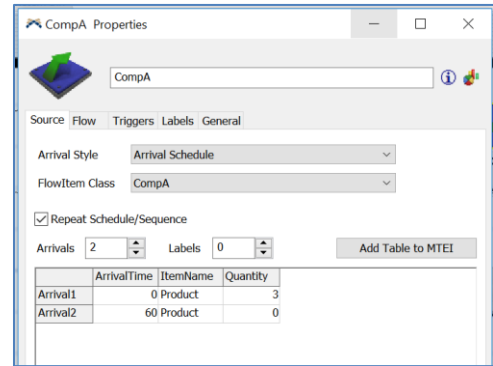
Each of the components need to be created by a Source.

- Create a Source for each component by dragging out two Sources, placing them near the packing station, and renaming them **CompA** and **CompB**.

The components arrive in batches, three per hour for CompA and five every half hour for CompB.

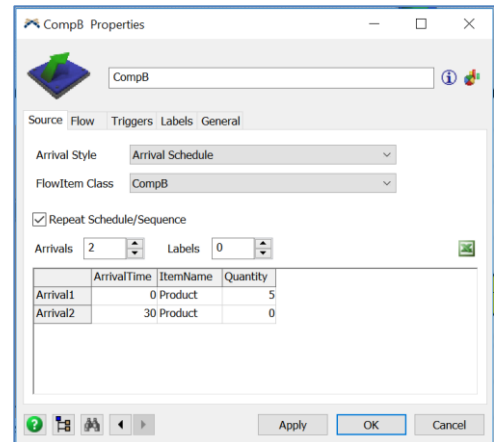
- Change the properties of the CompA Source to those shown in the figure to the right and described below.

- Change **Arrival Style** from the default **Inter-Arrival Time** to **Arrival Schedule** by using the drop-down menu.
- Change the **Flowitem Class** property from the default **Box** to **CompA** by using the drop-down menu.
- Check the **Repeat Schedule/Sequence** box.
- Configure the two-row **Arrival Table** as shown in the figure (**Arrivals** is 2 and **ArrivalTime** and **Quantity** as shown). This creates three flowitems at time 0 and three more flowitems every 60 minutes. This pattern will repeat since the box in the previous step is checked.



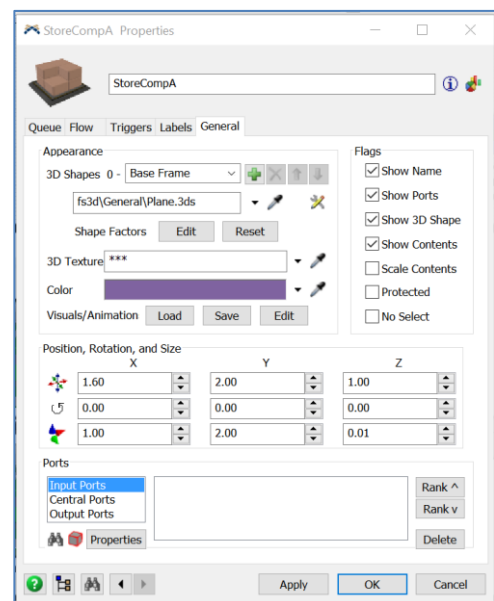
- Repeat the above steps for source CompB, as shown in the figure to the right and described below.

- Change **Arrival Style** from the default **Inter-Arrival Time** to **Arrival Schedule** by using the drop-down menu.
- Change the **Flowitem Class** property from the default **Box** to **CompB** by using the drop-down menu.
- Check the **Repeat Schedule/Sequence** box.
- Configure the two-row **Arrival Table** as shown in the figure (**Arrivals** is 2 and **ArrivalTime** and **Quantity** as shown). This creates five flowitems at time 0 and five more flowitems every 30 minutes. This pattern will repeat since the box in the previous step is checked.

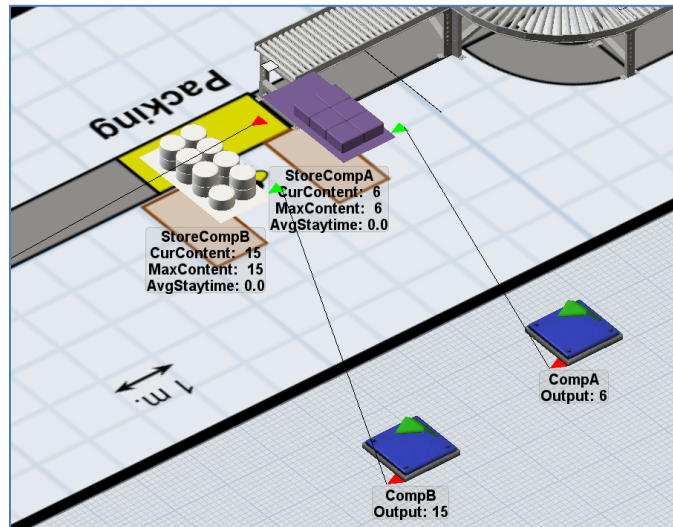


Add a Queue for each component to store the components. Since their properties are very similar, it is easier to set up one and then copy and paste it to create the other.

- Drag out a Queue object from the Library and place it near the Source.
- Modify the Queue's General tab as shown in the figure to the right and described below.
- Name the Queue **StoreCompA**.
 - Set the **3D Shape** to a **Plane**.
 - For the Color property, select purple from the drop-down menu so that the Queue is the same color as the component it stores.
 - Set each **Size** to x=1.0, y=2.0, and z=0.01.
 - Set the height, **z Location** not Size, to 1.0 so that is similar to the height of the conveyor.



- Locate the Queue on the layout so that it is in the space indicated on the side of the packing station.
- Copy and Paste the StoreCompA Queue by either of the following methods:
 - Select (click on) the Queue, press the **Ctrl-C** keys, click somewhere on the modeling surface, press the **Ctrl-V** keys.
 - Right click on the Queue, select **Edit** from the menu, then select **Copy**. Right click somewhere on the modeling surface, select **Edit** from the menu, then select **Paste**.
- Modify the General tab for second Queue, which will store Component Bs, as follows.
 - Change the name of the Queue to **StoreCompB**.
 - For the **Color** property, select white from the drop-down menu so that the Queue is the same color as the component it stores.
- Locate the Queue on the layout so that it is in the space indicated on the side of the packing station.
- A-connect each of the Sources to its respective Queue.
- **Reset** and **Run** the model. Validate that the arrivals of the components occur as expected, three CompA arrive every 60 minutes and five CompB arrive every 30 minutes. Below is a screenshot of the model just after one hour showing the expected queue contents – 6 in StorCompA and 15 in StorCompB.



- While changing Sources, change the **Flowitem Class** property on the ContainersArrive Source from Tote to **Container** so that it will generate the new flowitem that was created earlier.



If you haven't already done so, save the model. Recall, it is good practice to save often.

1.2 Modeling the packing operation using the Combiner object

Now that the components have been created, they can be packed into containers. This is done through *FlexSim's* Combiner object. As the name indicates it combines flowitems.

The Combiner object can operate in three modes, which is specified by the **Combine** property on the Combiner tab.

- *Pack*, the default mode option, allows items to be combined so they can be separated later. In this case, all flowitems received through input ports 2 and above are placed *in* the item that enters port 1, referred to in general as the container item.
- *Join* combines the items permanently. The flowitem entering port 1 is the only item that exits the object once all of the components have been gathered.
- *Batch* releases all items that are collected based on the quantities specified in the Components List.

The Separator object is the “opposite” of a Combiner – it unpacks items that are in a container that were packed by a Combiner. A Separator can also “copy” items; e.g., to simulate a cutting operation where single sheet of material enters a separator, but multiple pieces of that material exit by using the “Split” option on the Separator tab. The Separator object will be discussed further in a later section.



[section 5-4] Grouping and ungrouping flowitems.



If you have already done so, **Save** the model and then **Save As** Primer_3-2. Basically make a copy of the model so that it can be customized and the original model is retained.

In order to model the packing operation in this example, complete the following steps.

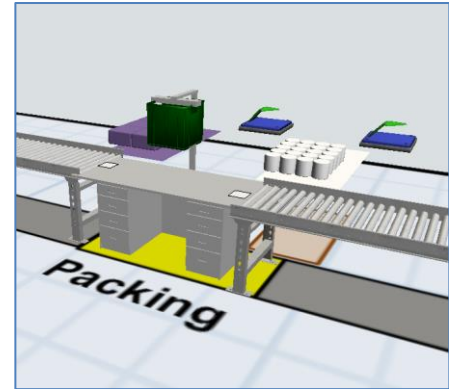
- Drag out a Combiner object from the Fixed Resources section of the Object Library onto the modeling surface.
- Name the Combiner, **Packing_1**.
- Disconnect the Straight Conveyor section just prior to packing, named StCv_ToPacking, from the Sink NextProcess. (Recall, using Q-disconnect)
- A-connect the Straight Conveyor section just prior to packing, named StCv_ToPacking, to the Combiner.
- Drag out a new Straight Conveyor section and then
 - Name the conveyor section **StCv_PackToNext**
 - Set its **Horizontal Length** to **8.0**.
 - Place the conveyor on the layout after the packing station.
- A-connect the Combiner to the new conveyor section, StCv_PackToNext.
- Connect the conveyor section, StCv_PackToNext, to the Sink NextProcess.
- One the General tab:
 - Change the Combiner's **3D Shape** property to the provided *SketchUp* file named **Workstation**.
 - Change the Combiner's size to x=**2.5**, y=**1.0**, z=**2.25**.
 - Change the **Color** property from yellow to **gray**.
- Locate the Combiner in the packing location on the layout, between the two Straight Conveyor sections.

- **Reset** and **Run** the model to verify that the basic flow is correct, as shown in the figure to the right. Of course the components are not yet packed in the containers. That is described below. However, before moving to the next step, it is important to make sure all of the objects are connected properly.

Upon closer examination, you may notice two visual problems with the packing station, as shown in the figure to the right:

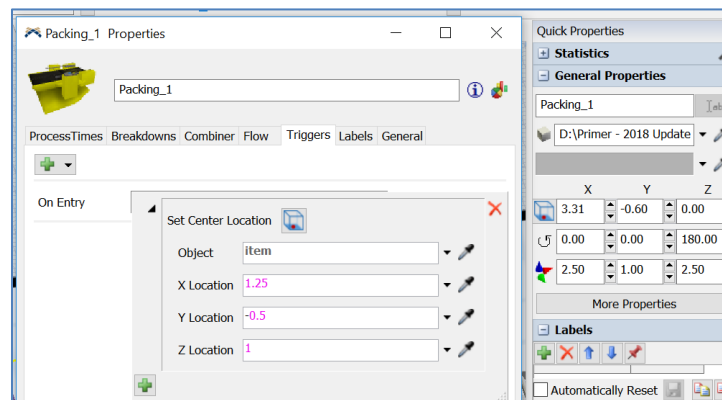
1. the container appears above the packing table and
2. the packing table is facing the wrong direction.

These types of effects can occur when importing shapes. While they don't affect model performance, they look odd.



The following changes correct these visual problems. The resulting property modifications are shown in the figure below.

- To fix the flowitem's position at the packing station, use the Combiner's **OnEntry** trigger and **Set Location** menu option, which is in the **Visual** category of the drop-down menu. Set the properties as shown in the figure below. Normally the property settings require experimenting with different values until the position looks right.
- To fix the rotation of the packing station, change its **rotation** to **180** degrees about the z axis as shown in the General Properties section of the Quick Properties interface in the figure below.

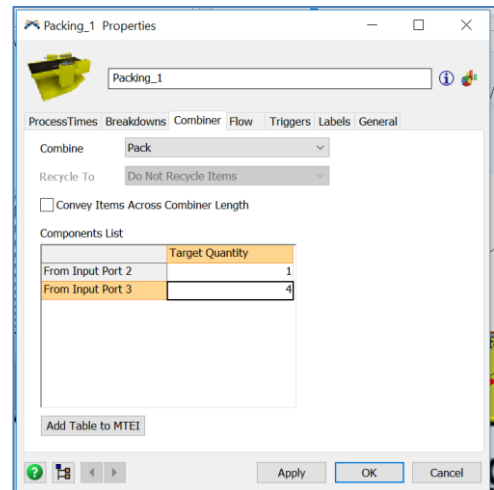


The two components that are to be packed are supplied to the Combiner via connections that are made from each component storage area (Queue) to the Combiner.

- A-connect the Queue StoreCompA to the Combiner.
- A-connect the Queue StoreCompB to the Combiner.

Each time a connection is made to a Combiner, a row is added in the **Components List** on the Combiner tab. The list is like a “recipe” for what to combine with the container. The **Target Quantity** is the number of items to collect from each port.

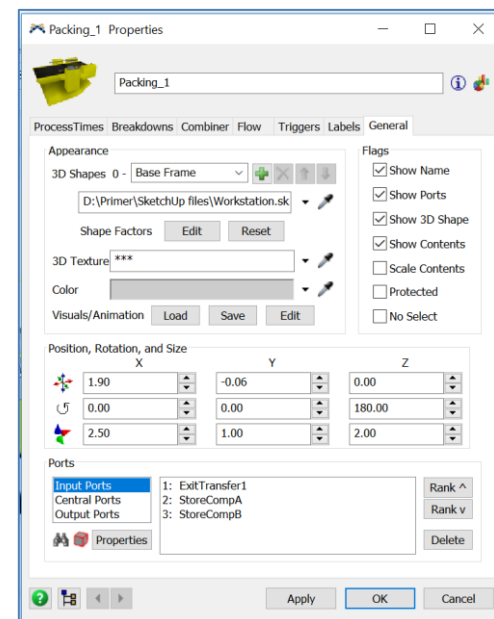
- Update the Components List as shown in the figure to the right. In this case, the Combiner is to collect **1** item from Port 2 (CompA) and **4** items from Port 3 (CompB). A single container always enters through Port 1.



One way to verify the port connections on any object is to examine the Ports section of the General tab. The figure to the right shows the input to the Combiner from ports 1, 2, and 3 are the conveyor (ExitTransfer1), StoreCompA, and StoreCompB, respectively.

Also, the ports (Input, Central, and Output) can be reordered by using the **Rank** controls and deleted with the **Delete** control, both are located to the right of the Ports list. If there are many connections to remove, it is much easier to do it from this interface than numerous Q-disconnects with the mouse.

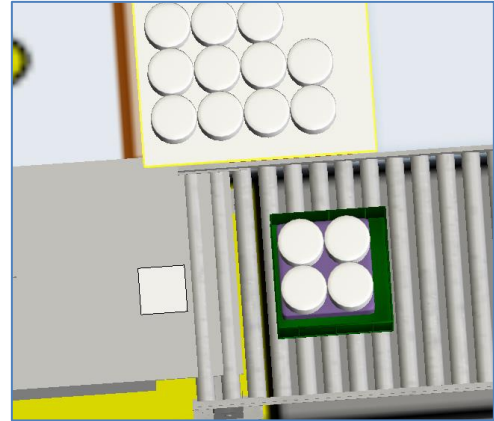
- Verify that the objects are connected properly and correct as necessary.



If you haven't already done so, save the model. Recall, it is good practice to save often.

Once the Combiner collects all of the items it needs from all of the ports, it invokes a process time that is specified in the **Process Time** property in the ProcessTimes tab. This tab is very similar to that tab in the Processor object. The process time represents the combining time, such as packing, assembly, etc. The default time is a constant 10 time units. The process time does not start until the **Target Quantity** for all of the items specified in the **Components List** have been collected.

- **Reset** and **Run** the model to verify that each container contains one CompA (purple box) and four CompBs (white cylinders). This is shown in the figure to the right.



The current model uses what is referred to as “fixed-recipe” combining; i.e., each type of container receives the same number and type of components. However, in this example the **Components List** on the Combiner tab needs to depend on the container type. Also, the process time, i.e., the packing time, should depend on the type of container.

The recipe for each container type is specified in a Global Table that is named Packing. The table, as shown in the figure to the right, has two rows and three columns. The rows correspond to the rows in the combiner’s **Component List**. In this case, they refer to Port 2 (CompA) and Port 3 (CompB). The columns represent the Type of the container. In this case, the container has Type values 1, 2, or 3. Therefore, the cells in the table are the **Target Quantities** for the Combiner. For this example, container Type 1 receives only two CompA, Type 2 receives only four CompB, and Type 3 receives one CompA and four CompB. Note the use of row and column headers (Row 1, Row 2, Col 1, etc.). Again, they are text fields that are useful for annotating the table - they are not used by the model, but clarify what information is stored in the table.

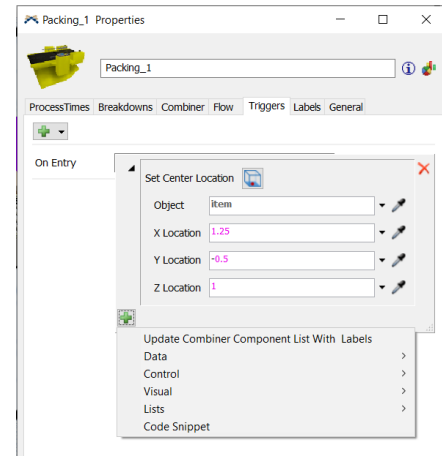
	Type 1	Type 2	Type 3
Port 2 - CompA	2	0	1
Port 3 - CompB	0	4	4

- Create a Global Table named Packing and edit the cells to correspond to the figure above.

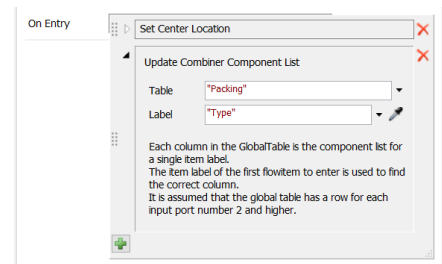
Notice that it would be very easy to expand this table to include many more product types (containers) than the three included in this example model. Also, as will be discussed later, the table could be imported from *MSEExcel*.

An **OnEntry** trigger on the Combiner is used to instruct the Combiner to use the recipes specified in the Global Table. Since the Combiner’s **On Entry** trigger is used to set the flowitem’s position, as was defined earlier, the + button on the **On Entry** trigger is used to add another **On Entry** trigger.

- Add the additional On Entry trigger to the Packing Combiner as shown in the figures to the right and described below.
 - Select the previously-defined **On Entry** trigger and press the green + in the lower left part of the interface.
 - Select the menu option **Update Combiner Component List With Labels**.



- As shown in the figure to the right, for the **Table** property, select **Packing** from the drop-down list of Global Tables.
- Retain the default value for the Label property, "Type."



Based on how the components are wrapped and prepared at the packing station, it is estimated that it takes 30 seconds to pack a component A and 20 seconds to pack a component B. In addition, about two minutes per container is needed to check and prepare the container, close it, and complete the necessary forms. Therefore, it takes 2.5 minutes to pack container Type 1, 3.33 minutes to pack Type 2, and 3.83 minutes to pack Type 3. In this case, based on the current product mix, the average packing time is 3.4 minutes. This is an average rate of 17.6 containers per hour ($60/3.4$). Since containers arrive at an average rate of 3/hour, the packing area is not expected to be a constraint.

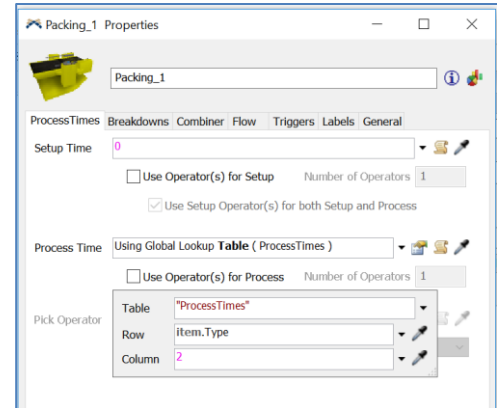
The packing times by container type that are defined above are used by the Combiner in the same manner as the finish times are used in the Processors - they are read from a Global Table. Since the format is the same, the packing times can be added as the second column in the **ProcessTimes** table.

- For the Global Table Process Times, add a column through the table's Quick Properties and update the cell values as shown in the figure to the right.

Model		
	ProcessTimes	ProductMix
	Finish Times	Packing Time
Type 1	15	2.50
Type 2	20	3.33
Type 3	30	3.83

Now the **Process Time** property on the Combiner is changed to use the values from the table created above.

- Change the Process Time property on the Combiner's Process Times tab from the default of 10 to the **By Global Table Lookup** option on its drop-down menu, as shown in the figure to the right.
 - For the **Table** property, select **ProcessTimes** from the drop-down list of Global Tables.
 - The default **Row** property is not changed since it uses the container's label Type for the lookup.
 - Change the **Column** property from 1 to **2** so that the values in the second column of the table are used.



- **Reset** and **Run** the model to verify that the containers are packed with the correct components and that the packing times vary by container type.



If you have already done so, **Save** the model and then **Save As** Primer_3-3. Basically make a copy of the model so that it can be customized and the original model is retained.

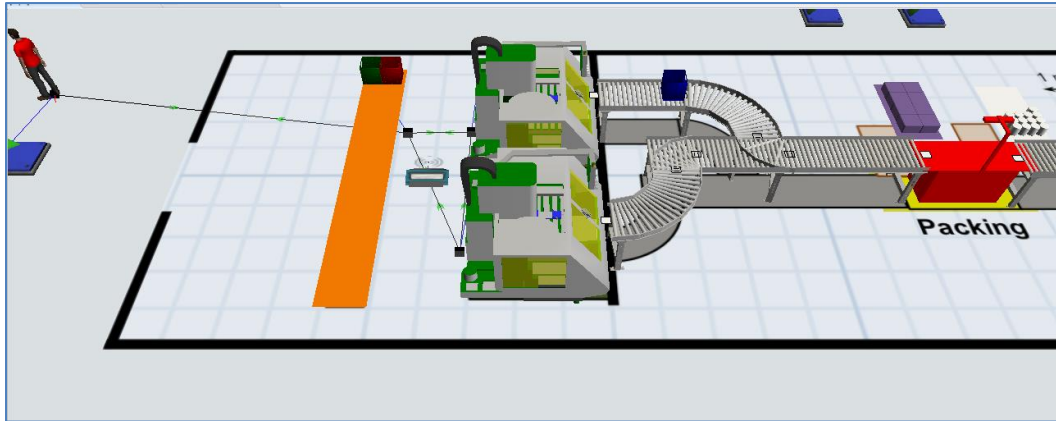
1.3 Downtime at the packing operation

In this model it is assumed that the packing operator is dedicated to a single packing station; i.e., the operator is not dynamic or a shared resource. Therefore, even though in the physical system there is a packing operator, an Operator object is not needed in the simulation. Of course, one could be added for visual effects, but it would not influence system performance.

Even though a packing operator is not explicitly used in the model, the downtime due to operator breaks should be considered. Therefore, the packing station, the Combiner object, is subjected to planned down time by using a Time Table. Since the packing area will follow the same downtime pattern as the finishing area, the Combiner can be added to the existing Time Table OperatorBreaks. However, in that table the operator travels to a location during the break, which is not possible in packing since it is considered a fixed resource for modeling purposes. Therefore, another Time Table is created, named OtherBreaks, that has all of the same properties as OperatorBreaks except the for the travel. Rather than recreate all aspects of the new table, it can be just be a copy of the previous table.

- Right click the OperatorBreaks Time Table in the Toolbox and select **Duplicate** from the drop-down menu. Modify the resulting Time Table OperatorBreaks_2 as follows:
 - Rename the Time Table, **OtherBreaks**.
 - On the Members tab, delete the **FinishOperator_1** object.
 - On the Members tab, add the **Packing_1** object (Combiner).
 - Change the **Down Function** back to the default by selecting **Stop Object** on the drop-down menu.
 - For the **On Resume** property change the **Color** property for the **Set Color (group)** option from **Yellow** to **Gray**.

As shown in the figure below, both the finishing operator and the packing area are on break since the objects are red. However, the finish machines continue to process containers and containers continue to arrive to the finish area. Of course the containers cannot be loaded onto the machines because the operator is on break. If upstream departments take a break at the same time, then the Source ContainersArrive could be added to the OtherBreaks Time Table.



Any finished containers that arrive at the packing area while the area is on break will wait on the conveyor until the packing area returns to work.

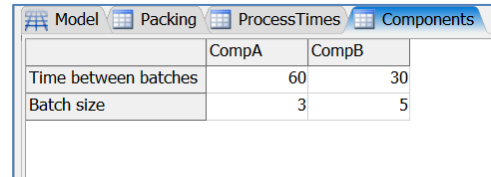


If you haven't already done so, save the model. Recall, it is good practice to save often.

1.4 Batches of components unloaded by the finish operator using Separator objects

Currently, the Sources for the components each generate a specified number of components to arrive at scheduled times. An alternative approach is to generate a single arrival, considered to be a batch or order, and then “split” or duplicate that item the number of times that is specified by the batch size. The model is now changed to take this approach.

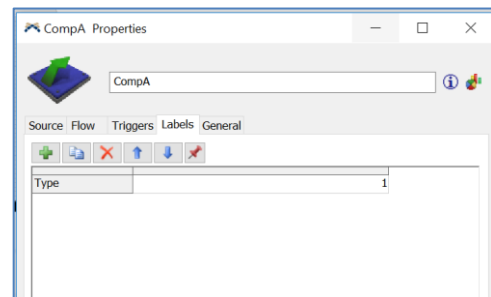
- First, set up some parameters in a Global Table named **Components**, as shown in the figure to the right. Each column corresponds to a component and each row is a parameter. For example, for CompA, every 60 minutes a batch of 3 items is delivered.



	CompA	CompB
Time between batches	60	30
Batch size	3	5

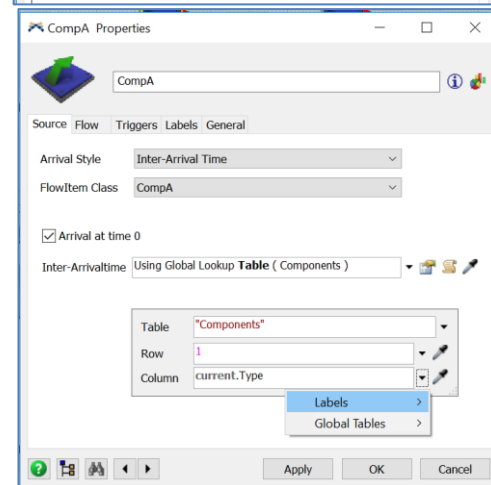
Prior to changing the arrival process, a **label** will be added to each component's Source in order to identify what type of item the Source is generating. Recall that a label is a characteristic of an object or item that may be accessed as a model runs. Also, recall that for flowitems the system automatically creates one label named Type.

- As shown in the figure to the right, the Source label is created on the Labels tab of each Source for the components.
 - Using the + button, select **Add Number Label**.
 - Change **labelName** to **Type**.
 - Set the value of Type to be **1** for Source CompA and **2** for CompB.



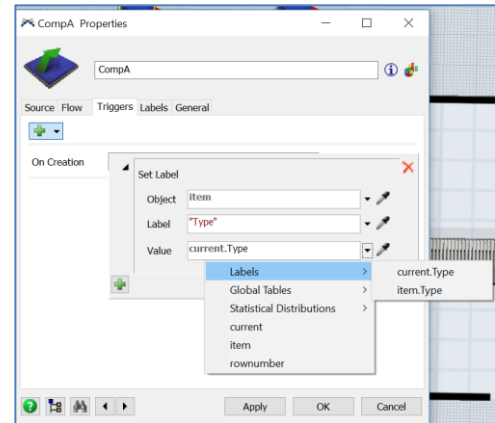
Each component's Source is modified so that it reads the time between batches from the Global Table Components.

- Modify CompA's Source as shown in the figure to the right and defined below. Also modify CompB's Source in the same manner.
 - The **Arrival Style** is changed back to **Inter-Arrival Time**.
 - The box **Arrival at time 0** is checked. This means that a batch is delivered when the simulation begins and all subsequent batches are delivered based the time between batches.
 - In the drop-down menu, the **Inter-ArrivalTime** property is changed to **By Global Table Lookup**.
 - For the **Table** property, the **Components** table is selected from the list of Global Tables that are in the model.
 - **Row** is set to **1** since the first row in the Components table is for the time between batches.
 - **Column** is changed to **current.Type** by selecting the **Labels** option in the drop-down menu and then **current.Type** on the subsequent menu. This means the column number that is used to obtain a value from the table is based on the label value named Type on the current object. For Source CompA, the lookup value is 1; whereas, the lookup value is 2 for CompB.



The items created by the Source will have the same Type value as the Source. This is accomplished as follows and as shown in the figure. Use this process for both Sources.

- Select the **On Creation** trigger on the Source's Trigger tab, then the **Data** category, and then **Set Label**.
- The **Object** property is the default value, **item**. The label is being set on the created flowitem.
- The value of the **Label** property is the default **Type**.
- The **Value** property is set by selecting **Labels** from the drop-down men, then selecting **current.Type**.



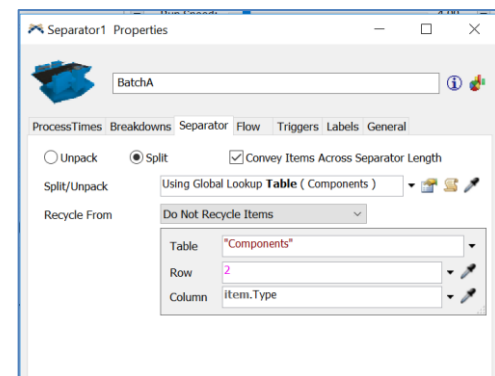
It is important to note that when referencing the label named Type on the *object*, **current.Type** is used; when referencing the label named Type on the *flowitem*, **item.Type** is used.

Notice that by using Global Tables it makes extending a model much easier and faster. Additional component Sources could be added by copying and pasting CompA or CompB and just changing their name and the value of the object's label Type.

The component Sources now generate one flowitem at a frequency determined by the time between batches. The Separator object is used to convert the single flowitem that represents a batch into the number of items in the batch.

The model needs two Separators, one for each component. One Separator will be created for CompA, as described below, and then it will be copied and pasted and the copied Separator will be modified for CompB.

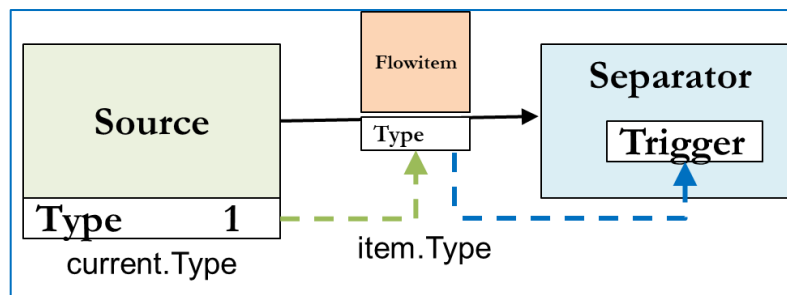
- Drag out a Separator object from the Fixed Resources section of the object and place it on the modeling surface near the packing area.
- Name the Separator **BatchA**.
- On the General tab,
 - Set its size to **x=1, y=1, z=0.05**.
 - Set its **z location** to **1.0** so that it is at the same height as the Queue. This makes it like a worktable.
 - Set the **Color** property to purple - the color of the flowitem it is processing.
- On the Separator tab,
 - Uncheck the property **Convey Items Across Separator Length**.
 - Click the **Split** button; note the default is **Unpack**.
 - As shown in the figure to the right, change the property **Split/UnPack** from the default **Entire Contents** to **By Global Table Lookup**.
 - For the **Table** property, select **Components** from the drop-down list.
 - Set the **Row** property to **2**. This is the row in the table that contains the values for batch size.



- Set the **Column** property to **item.Type** by selecting **Labels**, then **item.Type**. The column in the table is the value of the label stored in the entering item's label named Type.
- Copy and Paste Separator BatchA by: selecting the Separator, pressing Ctrl-C, clicking on the modeling surface, and pressing Ctrl-V.
 - Change the following on the pasted Separator. All other properties created above pertain to both objects.
 - Rename the Separator **BatchB**.
 - Change the color to white.
 - Integrate the Separators into the model.
 - Delete the connection between each component Source (CompA and CompB) and the Queue (StoreCompA and StoreCompB).
 - Connect each Source to their associated Separator, e.g., connect CompA to BatchA.
 - Connect each Separator to their associated Queue, e.g., connect BatchA to StoreCompA.
 - Position the Separators on the layout adjacent to their associated Queues, e.g., BatchA near StoreCompA.

Note that the time to split batch is 10 minutes since the default Process Time property is used in the Separator. That is okay for now since it will be changed shortly.

The use of labels in this example is summarized in the figure below. Each Source has a label denoting the type of component it generates; the name of the label is Type. Whenever a flowitem is created by the Source, the value of its label named Type is assigned to a label on the flowitem, in this case it is also named Type. Basically the operation $\text{item.Type} = \text{current.Type}$ is performed. The Separator then uses the flowitem's label value to determine the number of items that are in a batch.



The time for a finish operator to split a batch is estimated to be one minute plus 3 seconds per item. The number of items in a batch is stored by component type in the second row of the Components table. Since the fixed time per batch and the time per item might change, or vary by product type, it is a good idea to put these in a table as well. Therefore:

- Add two rows to the Components table for these parameters, as shown in the figure to the right. While the fixed and variable times to unpack a batch may be the same for all component types, adding these parameters to the Components table provides the flexibility to vary the values by type. For example, some items might be more difficult to handle based on their size or may require special handling.

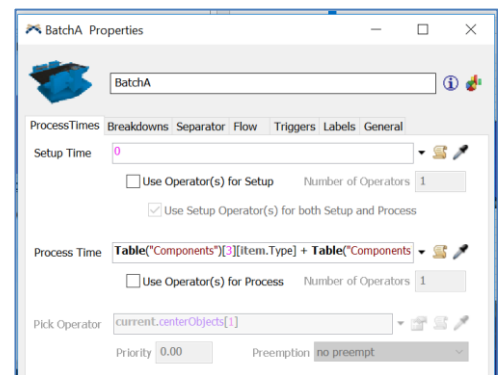
	CompA	CompB
Time between batches	60	30
Batch size	3	5
Time to unload a batch - fixed (minutes)	1	1
Time to unload a batch, per item (minutes)	0.05	0.05

The **Process Time** at each Separator, to unpack a batch, is expressed as the following.

```
Table("Components")[3][ item.Type]
+ Table("Components")[4][ item.Type] *Table("Components")[2][item.Type]
```

- Enter this expression carefully and exactly as it appears because this actually a programming statement. It all must be entered on one line; the statement above is broken up across multiple lines for readability. Implementation of the expression is shown in the figure to the right.

As the expression is typed in the text box, *FlexSim's* context-sensitive help makes it a bit easier by suggesting command names and table names and when a leading bracket or parenthesis is entered, the ending symbol is automatically provided by *FlexSim*.



Be careful on entering (or [- one is a parenthesis and the other is a bracket and they have different meanings in computer programming.

Also, copying and pasting can make entering the statement a bit easier; e.g. the first term can be copied and pasted and then edited, as opposed to typing in each character.

Once typed into one Separator, the expression can then be copied and pasted into the other Separator.

The Process Time expression that is defined above is interpreted as follows. In general, the fixed time per batch is added to the product of the time per item and the number of items in the batch. In this case, the fixed process time per batch is the value obtained from the cell in the Global Table Components that is located in row 3 and in the column number that corresponds to the type of the entering flowitem. Similarly, the process time per item and the number of items in a batch are the values obtained from the cells in the Global Table Components that are located in rows 4 and 5 respectively and in the column number that corresponds to the type of the entering flowitem.

While this operation may seem quite complicated, it demonstrates the power and flexibility of *FlexSim*. This is actually a coding or programming statement, albeit a simple one. A constant time could have been used, and that might be a good strategy early in the model-building process. However, one goal of this primer is to introduce, or at least illustrate, some of the power of *FlexSim*.

It was mentioned earlier in the primer that one way to customize objects and logic in *FlexSim* is via scripting in a C++ type of computer programming language called Flexscript. The expression above is a Flexscript statement. Other Flexscript statements and commands have been used in the primer, but they have not been

identified as such. For example, the statistical distributions - such as **triangular(min, max, ml, getstream(current))**, **duniform(min, max, getstream(current))**, **dempirical("tableName")** - are all examples of Flexscript commands that are programmed to obtain random samples from the designated distribution. Also, **item.Type** is Flexscript's reference to the value of the label named Type on the flowitem that is being processed. More information on coding in Flexscript is available in the *FlexSim User Manual* and the following:

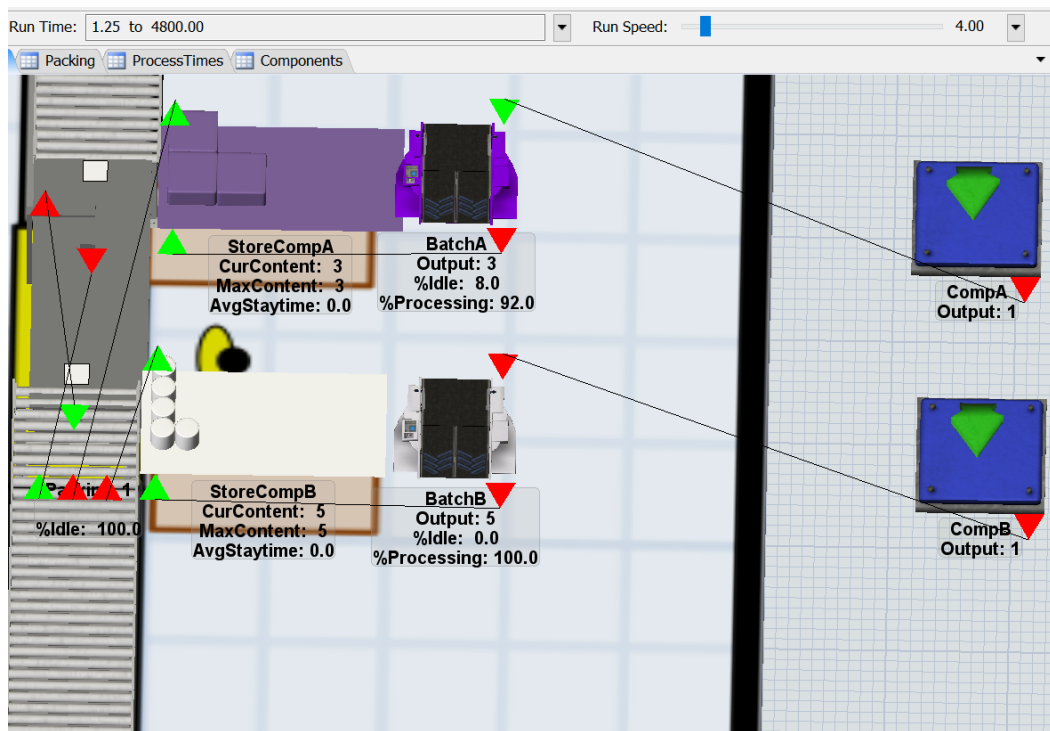


Another primer by this author, entitled *A Primer on Coding in FlexSim 2017*.



[Section 12-3] Scripting in *FlexSim*

- **Reset** and **Run** the model and verify that batches arrive when scheduled and the Separators split each batch into the specified batch size. In the figure below, the batch of three CompA entered the queue at simulation time 1.15 minutes ($1 + 3 * 0.05$); similarly, the batch of five CompB entered its queue at time 1.25 ($1 + 5 * 0.05$). Thus, the Process Time expressions in the Separators are working correctly.

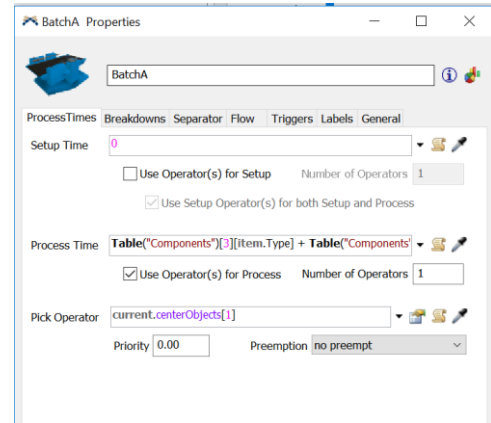


If you haven't already done so, save the model. Recall, it is good practice to save often.

The finish operator must perform the operation at the Separator, i.e., unload batches.

- As shown in the figure to the right, check the box **Use Operator(s) for Process** on the ProcessTimes tab of each Separator, BatchA and BatchB. This sends a request for an Operator to travel to the object and be delayed by the value of the **Process Time** property - in this case the time to unload a batch of components.

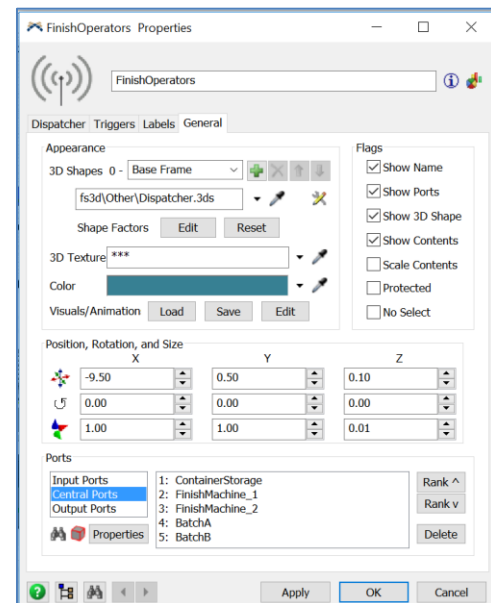
Note that the **Pick Operator** property value is grayed out until the **Use Operator(s) for Process** box is checked. In this case, the default **Pick Operator** property is used and the result is that the request to perform the task is sent to the Dispatcher, then the Dispatcher sends the request to an attached Operator. Of course, in this case there is just one finish operator.



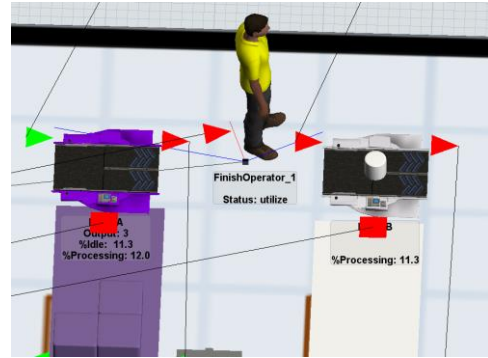
Once the process time is complete the finish operator is free to perform another task; it becomes idle and waits at the batch area until it receives another task. This is referred to as the basic **utilize** task sequence and its tasks differ from the basic **transport** tasks described earlier. This task sequence is composed of just two tasks: (1) travel to the object and (2) be utilized for the specified time. As mentioned in a previous section, the task sequences can be customized, but that is a more advanced feature of *FlexSim*.

Of course each Separator must be connected to the Dispatcher.

- Use a center- (or S-) connection to connect each Separator and the Dispatcher. Notice, as shown in the figure to the right, the Dispatcher is now connected to five objects; i.e., it receives tasks from each of these objects.



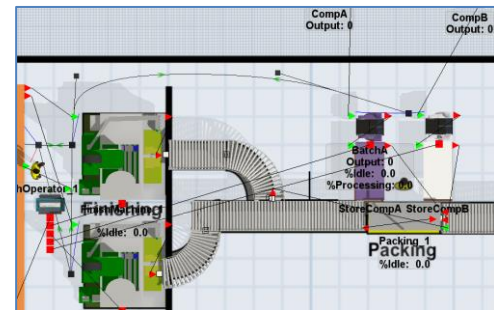
In a previous part of the primer the Operator was placed on a network in order to restrict its travel paths. Therefore, for the operator to do the unpacking of the component batches, the network needs to be extended to include the packing area. Otherwise, an error will occur since the Operator object will not know how to reach the area to unpack the batches by staying on the network.



As shown in the figure to the right, only one Network Node is added to the Operator's path network. This is because in the next section an alternative means for controlling the travel paths of Task Executors is discussed. Note in the figure that both Separators use the same Network node and the Operator performs the unpacking operation at the Node between the two Separators since the Operator has been instructed not to travel offsets.

To enable the Operator to travel to the packing area:

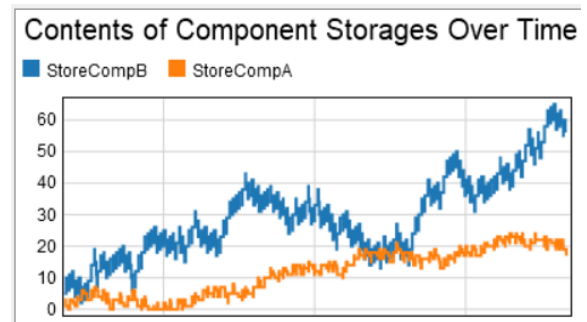
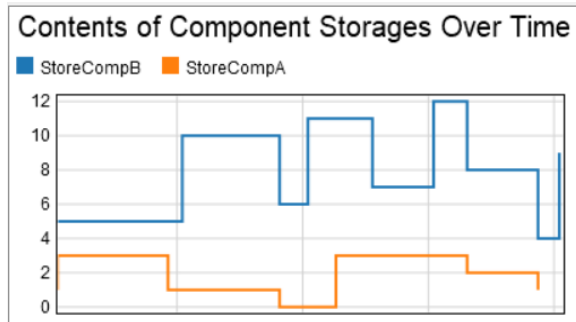
- Drag and drop a Network Node from the Object Library and place it between the two Separators.
- Name the node, **nn_UnBatching**.
- A-connect the node to nn_ContainersArrive.
- A-connect the node to both Separators, BatchA and Batch B.
- Optionally, as shown in the figure to the right, reshape the path so that the Operator does not travel the straight-line route between the finishing and packing areas and travel through FinishMachine_1 and other objects. To do so:
 - Right-click on one of the green arrows on the just-completed path and select **Curved** from the menu.
 - As a result of the above selection, two black squares, referred to as “handles,” appear on the path between the green arrows. Use the handles to define a smooth path that avoids any obstacles, as shown in the figure to the right. You might need to initially place the Node away from the area so that you can access the two handles since they may be hidden by another object.



It would be informative for analysis to add a Dashboard that displays the contents of the component queues over time.

- Create a new Dashboard by using the **Add Dashboard** drop-down menu and name it **Component Storage**.
- Select the **Content** chart template from the Content section of the Dashboard Library and then select Line Chart. Drag the template to the new Dashboard.
- Name the chart, **Contents of Component Storage over Time**.
- Using the + button in the **Objects** section of the Options tab, add two objects, both component Queues, StoreCompA and StoreCompB.
- On the Settings tab, change the **Time Access Mode** to **Show Duration** and set the time unit to **Hours**.

- **Reset** and **Run** the model. The contents of the component storages should be similar to the plots shown in the figure below. The plot on the left is after about two hours (120 minutes); the plot on the right is at the end of the simulation run of 4,800 minutes (80 hours).



Based on the single 80-hour simulation run, shown to the right above, it appears that the trend in the number of components waiting to be packed continues to grow over time. Therefore, the time between batches and/or the batch size should be adjusted. The simulation model can be used to decide the best settings. Also, in the model there is basically no limit on the number of components that can be stored prior to packing. The physical space limitations need to be considered.



If you haven't already done so, save the model. Recall, it is good practice to save often.

2 CONTROLLING TASK EXECUTER TRAVEL PATHS USING A*

By default, Task Executors, such as Operators and Transporters, travel in straight lines between points in a model. While this is efficient, it can be unrealistic – in reality operators cannot go through machines, conveyors, etc. The straight-line path not only doesn't look right, it can affect estimated system performance – it takes longer to follow an object-avoidance route compared to the straight-line distance between objects.

Up to this point in the primer models, the travel paths of the Task Executors, i.e. the finish operator, is controlled through a network of connected Network Nodes. Each Node is considered an object and is placed on the modeling surface at key locations to establish the paths, which can have either straight or curved segments. In essence, this approach tells the Task Executor where to go. The alternative approach that is considered here, referred to as A* or AStar, tells the Task Executor where *not* to go and let's the TE decide the path that avoids the specified obstacles.

A* (pronounced A-star) is a popular search algorithm that is commonly used in computer science and mathematics. It uses heuristics to find the shortest path between points considering barriers or no-travel zones. As with all other topics in the primer, only the basics are introduced here and mostly the default settings are used.

Utilizing the A* algorithm in *FlexSim* is quite easy. The objects associated with the A* algorithm are in the section of the Object Library named A* Navigation, just below the Visual section.

2.1 The basics of the A* algorithm using a simple study model

Before implementing A* in the current primer model, use a simple study model to understand the basics.

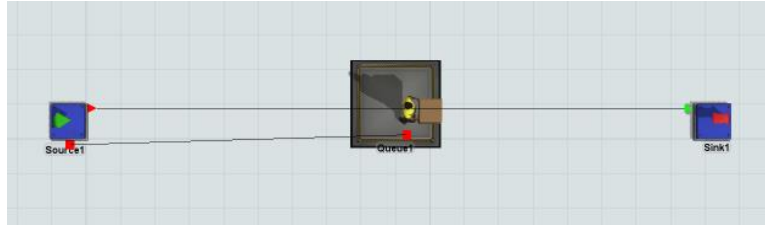
- Create a new model using all default settings.



It is good modeling practice to try new concepts outside of the main model being constructed. These simple models, often referred to as *study* models, are useful in learning new concepts or understanding how *FlexSim* features behave. They focus on a particular problem or a portion of complex logic.

As shown in the figure below, create a very simple study model of an operator transporting items between a Source and a Sink. Use all default values. Add a Queue that is not connected to any other objects – it is used only to represent a storage area that the operator should avoid.

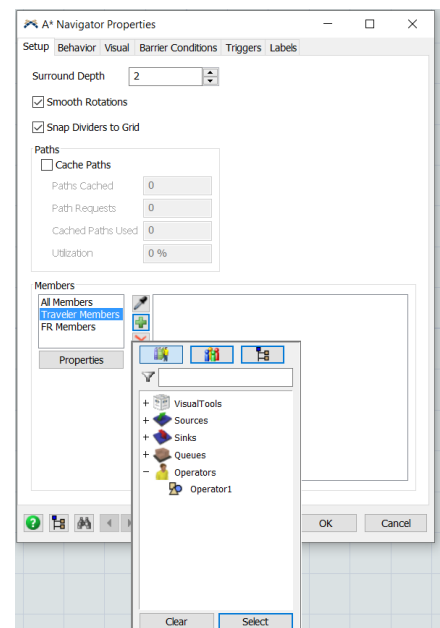
- Drag out a Source, Sink, Operator and Queue from the Object Library.
- A-connect the Source to the Sink.
- Reset the view so that it is in planar view.
- Place the Queue on the straight-line path between the Source and Sink.
- Center-connect the Operator to the Source.
- On the Flow tab of the Source, check the property **Use Transport**.



- Save the model as A-StarStudy.
- **Reset** and **Run** the model. As expected the Operator's path should be through the Queue.

Now, use the A* tools to avoid the queue and other barriers. The tools are located in the A* Navigation section of the Object Library, right after the Visuals section.

- Drag out the A*Navigator object and place it anywhere on the modeling surface.
- Double click the A*Navigator object and note it has Setup, Behavior, Visual, and Barrier Conditions tabs in addition to the common FlexSim object tabs Triggers and Labels.
- In the **Members** section of the Setup tab, use the + button to add objects to the **Traveler Members** list. In this case, select the **Operator1** in the Operators section, as shown in the figure to the right. Objects selected here will have the A* algorithm applied to them.



- In a similar manner, add the **FR Members**, where FR means Fixed Resource. The interface is just below the **Traveler Members**, again on the Setup tab.
 - Select the **Queue** since this is the object the Operator needs to avoid.
 - By selecting an object category, e.g. Queues, all objects in that category are selected. Of course, in this case, there is only one object in an object category.

- **Reset** and **Run** the model. Note, as shown in the figure to the right:
- A blue box provides an outline of the objects that are considered by A*. The box can be hidden by unchecking **Show Bounds** on the Visual tab of the A*Navigator object.
 - The Operator now avoids the Queue. By using A*, the Operator takes the shortest path between the Source and Sink that avoids the Queue.
 - The size and location of the blue box may need to be adjusted since the operator will not travel beyond its boundaries. It can be repositioned by selecting anywhere on the blue border and dragging the box to the desired location. It can be resized by selecting and dragging one of the red arrows that appear on the border when the object is selected.

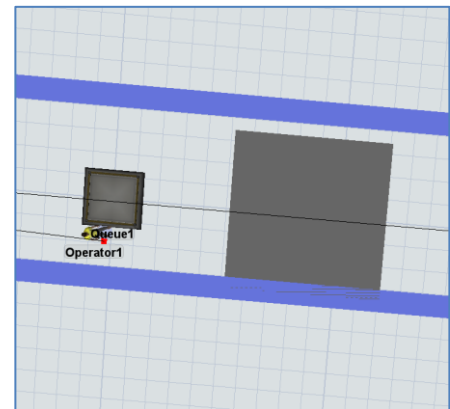


If you haven't already done so, save the model. Recall, it is good practice to save often.

TEs can avoid objects that are not basic Fixed Resource objects. Two examples are illustrated here. A *Barrier* object is a rectangular shape that will cause a TE to move around the object; it can be located anywhere in the 3-dimensional space. A *Divider* object is a set of line-segments in 3-dimensional space that creates either a one-way or two-barrier. The line segments are analogous to walls.

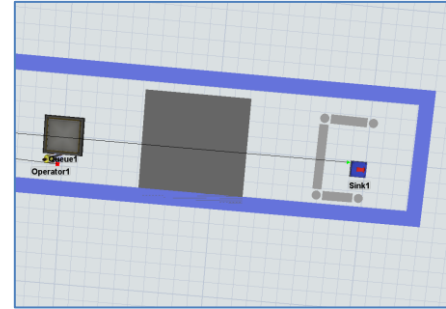
- As shown in the figure to the right, drag out a Barrier object from the Object Library and place it on the modeling surface. Its size and location can be changed by double clicking the object or via Quick Properties.

The barrier could be used to represent a storage area instead of using the queue object.



As shown in the figure to the right, a Divider object is added around the sink:

- Select the Divider object from the A* section of the Object Library and then click on the layout where a divider should start. A gray circle marks this spot.
- Use the mouse to draw the Divider, setting its direction and length. Click at an end point of the Divider and another circle marks this point. Continue this process until the desired shape is drawn on the layout.
- Click the Esc key to stop drawing the Divider.
- The Divider can be repositioned and reshaped by:
 - Selecting one of the line segments and moving the entire divider object, retaining its shape and size.
 - Selecting and moving a circle segment to change the dividers shape and size. Moving a circle changes the length of the line segment and/or the angle between the segments.

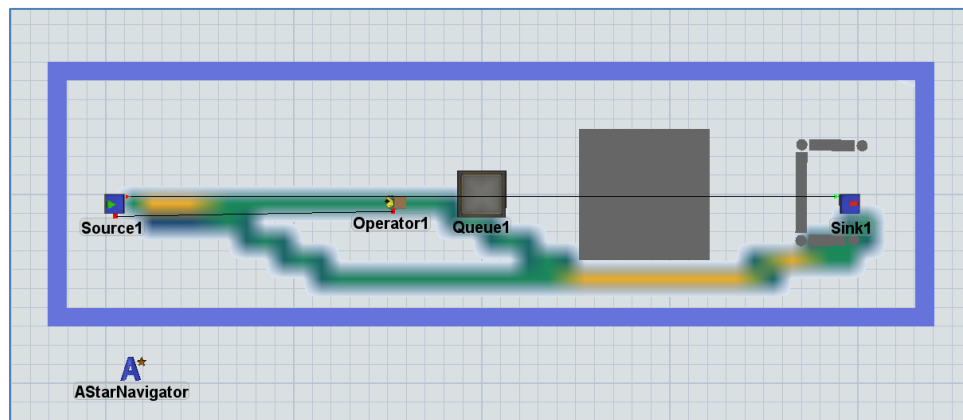


- **Reset** and **Run** the model and observe the behavior. The Operator should identify and follow a path between the Source and Sink that avoids the Barrier, Divider, and Queue.

A heat map can be used to see the path the operator travels and the frequency the path is used.

- Check the **Show Heat Map** box on the Visual tab. Also, just below this property, check the **Transparent Base Color** box.

The results are shown in the figure below. Initially, notice that the operator chooses a slightly different path when going to the Sink from the Source and to the Source from the Sink. The “warmer” zone, which indicates a higher travel frequency on that path, is in yellow, as opposed to green.



If you haven't already done so, save the model. Recall, it is good practice to save often.

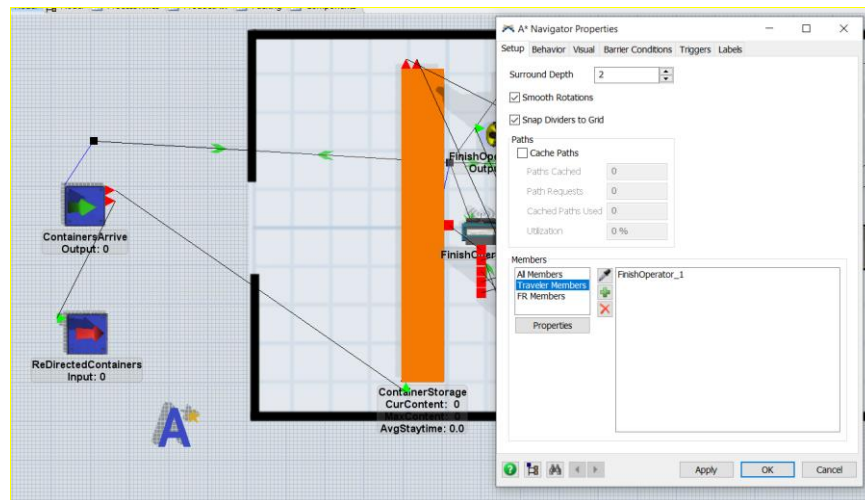
2.2 Implementing the A* algorithm in the primer model

Now that the basics of using the A* algorithm is introduced through a simple study model, it is added to the primer model.



Begin with the basic model from the previous section, named Primer_3-3, and **Save As** Primer_3-4. Basically make a copy of the model so that it can be customized and the original model is retained.

- Drag out the A*Navigator object anywhere on the modeling surface outside of the layout, as shown in the figure below.
- On the Setup tab and in its **Members** section, use the + button to add objects to the **Traveler** **Members** list. In this case, select the **FinishOperator_1** in the Operators section, as shown in the figure below. Objects selected here will have the A* algorithm applied to them.



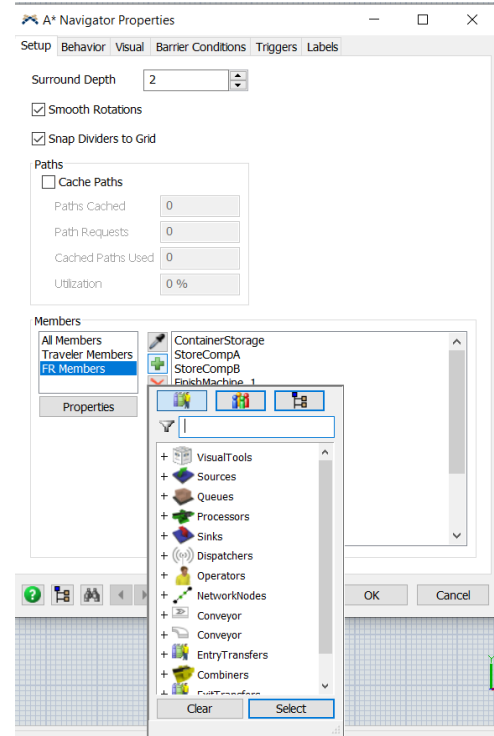
In a similar manner, add the **FR Members**.

As shown in the figure to the right, the interface is just below the **Traveler Members**, again on the Setup tab.

- Select all **Queues, Processors, Conveyors** (Straight and Curved), **Entry Transfers, Combiners, Exit Transfers,** and **Separators**. These are the objects that are used by A* and the objects that the Operator needs to avoid.
- By selecting an object category, e.g. Queues, all objects in that category are selected. In this case, by selecting the Queues category, the objects ContainerStorage, StoreCompA, and StoreCompB are added to the FR Members list.

Multiple object categories can be selected at a time by holding down the Shift-key when selecting categories.

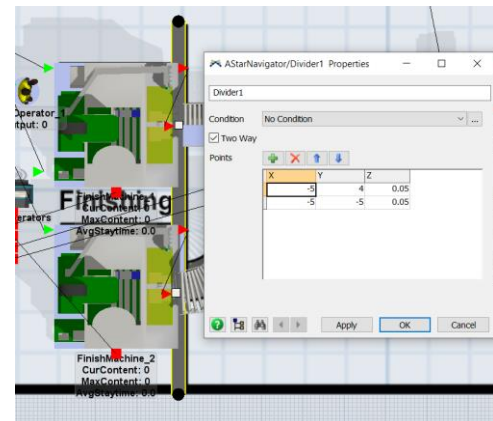
All of the object types in the model are selected except Visual Tools, Sources, Sinks, Dispatchers, Operators, and Network Nodes – these are not considered to be barriers to the Operator's travel.



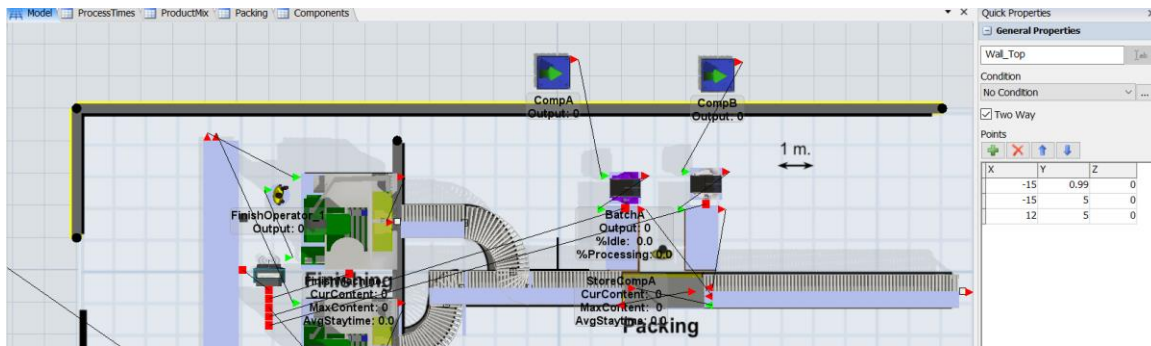
Besides the basic *FlexSim* modeling objects that are used in the model, the only other thing the Operator should avoid are the facility's walls. While it is not necessary to include all of the walls on the layout, they are added in this case to get practice with the A* tool.

The Operator needs to avoid the walls in the system. The Divider tool is used to add walls to the model.

- It is best to zoom out so that the entire layout is visible on the screen. Also, it is suggested to Reset the view to the planar or two-dimensional view. (Recall, this is done by right clicking anywhere on the modeling surface and selecting **View**, then **Reset View**.)
- Select the Divider object from the Object Library and click on one end of the vertical wall to the right of the finish machines. (This wall divides the finish area from the packing area). Then click on the other end of the wall on the layout. Press the Esc key to stop drawing the divider. This results in a line segment with a circle at each end, as shown in the figure to the right.
- Adjust the shape as necessary by dragging a circle end point or the line segment. The wall can be located more precisely by double clicking the Divider object and modifying the table values, as shown in the figure to the right. Each row in the table is the x-y-z coordinate of one of the circles on the Divider. This table may also be accessed in Quick Properties when the object is selected.
- Change the name of the object from Divider1 to **WallBetweenFinishAndPack**.



- Select the Divider object again and click on the end of the vertical wall at the left end of the layout at the opening. Then, click on the wall corner (adding a circle) and finish the wall at the right-hand side of the layout. Press the Esc key to stop drawing the divider. Adjust the shape as necessary by dragging a circle end point or the line segment or modifying the table in Quick Properties, as shown in the figure below. Also, name the object **Wall_Top**.



- Again, similar to the step above, select the Divider object and click on the end of the vertical wall at the left end of the layout at the other end of the opening. Then, click on the wall corner (adding a circle) and finish the wall at the right-hand side of the layout. Press the Esc key to stop drawing the divider. Adjust the shape as necessary.
- On the Visual tab of the A* object, check the **Show Heat Map** box and the **Transparent Base Color** box.

The Operator will now be controlled using the A* algorithm and not the path network created earlier.

- Therefore, zoom in toward the Operator. Disconnect the operator from the network by holding down the Q key, selecting the operator then dragging and selecting the network node where the operator is connected to the network (red line). If this is done correctly, the red line between FinishOperator_1 and nn_Container Storage should be removed.

Note that barriers and dividers have no height; the z-value is only for location of the barrier/divider in the z direction. To represent a wall in a model, use the Walls object in the Visual section of the Object Library.

All of the network nodes for the path could be deleted, but in this case, they are left in the model.

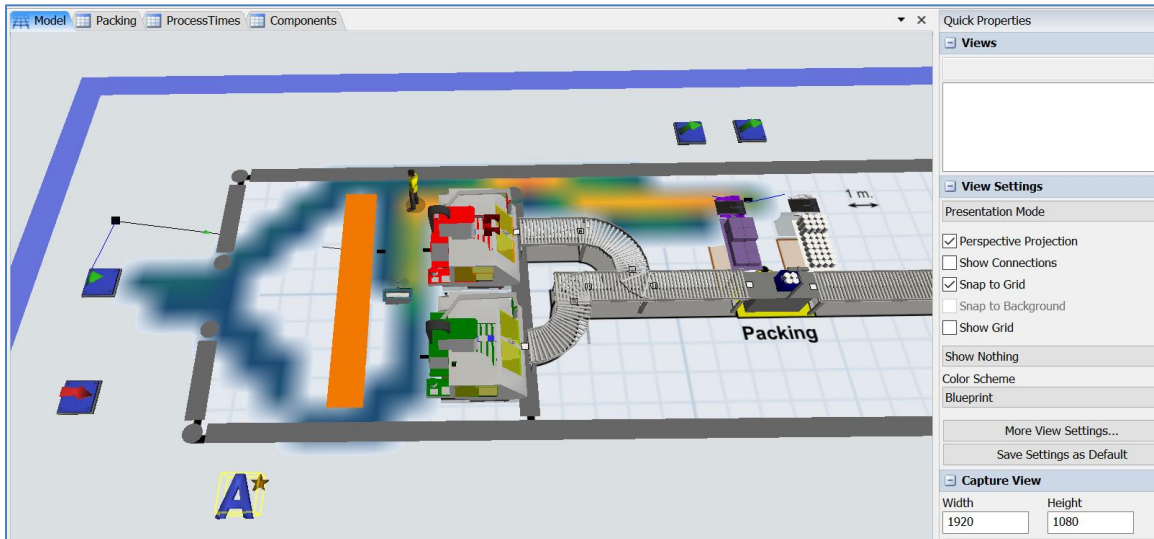
- **Reset** and **Run** the model. Verify that the Operator avoids all of the fixed resources, barriers, and dividers, as shown in the figure below.

➤

In the following figure, all of the connections are removed from view; this is accomplished by:

- Click anywhere on the modeling surface. In the Quick Properties window, change the **View Settings** property from **Working Mode** to **Presentation Mode**.

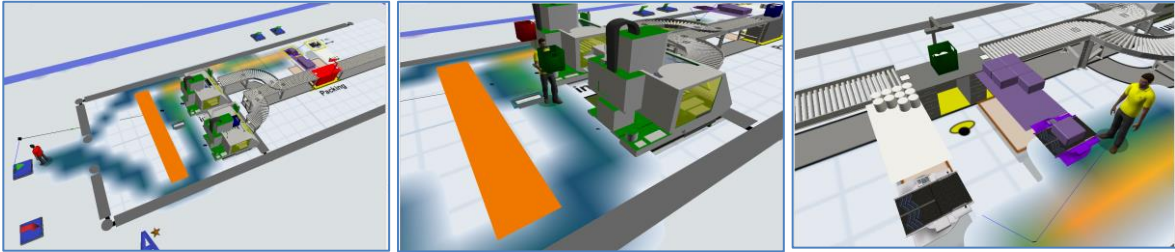
For clarity, you may want to uncheck all of the boxes on the Visual tab of the A* object. Of course this does not affect the underlying logic, it just makes the model a bit cleaner.



If you haven't already done so, save the model. Recall, it is good practice to save often.

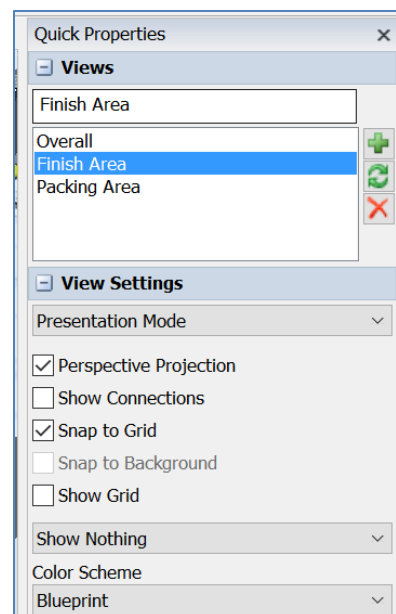
2.3 Saving model views

While not related to the A* algorithm, it is an opportune time to introduce the ability to set and save multiple model views. With saved views, various aspects of the model can be examined quickly. The following three views will be created and saved.



Model views are defined in the **Views** section of the Quick Properties window. As shown, in the figure to the right, the three model views that are listed – Overall, Finishing, and Packing – are for the views shown in the figures above.

- Define each of the three views by orienting the 3D view of the model to the desired look, then pressing the + button in the Views section of the Quick Properties window.
- Each view should be renamed to a meaningful name, e.g. Overall.



Views can be accessed anytime by clicking on the desired view in the Views section of the Quick Properties window.



If you haven't already done so, save the model. Recall, it is good practice to save often.

3 USING AN ITEM LIST FOR COMPLEXMODELING LOGIC

In the finishing area of the primer example, the current model uses the default logic in the Queue object to determine the order in which containers are processed. That default logic is first-come, first-served (FIFO); i.e., items are processed in the order in which they arrive. In other words, the items that have waited the longest are processed first. This is a common ways for queues to operate; many human systems operate this way because it is perceived to be the fairest.

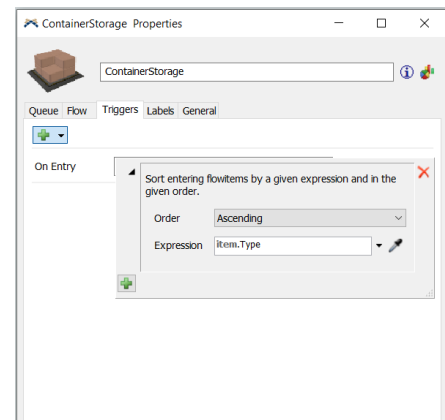
However, there are exceptions to this means of processing, such as when items are processed by priority. For example, patients in a hospital waiting room do not follow the FIFO rule - patients are triaged based on the seriousness of their situation. Also, customer orders are oftentimes processed before those that have waited longer, if they are designated as close to a due date or late, or if the customer paid a premium to get the order processed quickly.

3.1 Simple priority routing using an object trigger

In the primer example, the desire is to process containers by using the shortest processing time (SPT) rule – process the quickest items to process first. In production control theory, processing by SPT results in more throughput than by using FIFO. Based on the process times, the preferred order for finishing containers is: Type=1 first, then Type=2, and then Type=3. (Recall, finish times are 15.0, 20.0, and 30.0 minutes for types 1, 2, and 3, respectively.)

To represent this in *FlexSim* is quite easy. As shown in the figure to the right, the contents of the Queue can be reordered whenever an item enters.

- On the Triggers tab of the Queue ContainerStorage, add an **OnEntry** trigger via the + drop-down menu.
- On the OnEntry trigger, select the **Control** category, then **Sort by Expression** in the drop-down menu.
- As shown in the figure to the right, change the **Order** property from the default **Descending** to **Ascending** using the drop-down menu. Ascending is chosen because the Type=1 containers should be processed first, then Type=2, etc. ; i.e., items should be arranged in the Queue by increasing/ascending values of the label Type.
- The default **Expression** property, **item.Type**, is used as the sort criteria; i.e., the value of the label Type on the items in the Queue. If there are multiple items of the same type in the Queue, they are arranged in the order in which they arrived. Therefore, there are basically two ordering rules: first, order by type, then by age.



Note that this approach changes the order of the items in the Queue – the contents of the Queue is sorted each time an item arrives to the Queue and the item at the front of the queue is sent to the first available object (a Processor in this case). Thus, containers are finished (processed at the Processors) based on the SPT or priority rule. Many systems behave this way.

However, other systems maintain the order in which items arrive and then items are selected from the Queue based on the SPT rule. This would be the case if items arrive on a conveyor - items maintain the order in which they arrive since they cannot physically be reordered. While in many cases this is a subtle difference, it can affect performance. For example, in this case, if a priority container is the last to arrive, then the finish operator needs to travel to the end of the queue in order to load the container, rather than taking it from the front of the queue, resulting in a longer travel time. Of course, the level of detail is always an important modeling decision. In many cases, it might be best to use the quick change (**OnEntry** trigger to reorder the Queue), make and note an assumption that this does not significantly impact performance, and then revisit the assumption later.

The use of the List tool, described in the next section, does not reorder the queue, it just identifies the next item to process based on a specified rule.

3.2 Multiple-criteria routing using Lists

One drawback of using the SPT rule in this example is that the Type 3 containers may wait a very long time. This may occur because if there are any Type 1 or 2 items in the Queue when a finish machine becomes available, then they will be processed before any Type 3. This issue is addressed by using the following rule: *process containers based on Type unless some items have waited “too long,” then process them first.* Of course, “too long” must be specified, such as one hour; this is referred to as the *wait threshold*. The List tool is used to implement this more complex rule.

Prior introducing Lists, two new constructs, “pull” logic and “push” logic are defined.

- **Push logic.** In the current model, items are “pushed” from the Queue to the Processors (from ContainerStorage to FinishMach_1 and FinishMach_2); i.e., the routing decision is in the Queue and it pushes items to the first available Processor by default or by a rule specified in the **Send To Port** trigger on the Flow tab of the Queue. The default rule is **First Available**, but many others are available on the drop-down menu, such as **Random**, **Round Robin** (alternate between machines), **By Expression** (using the value of a label, such as Type), etc.
- **Pull logic.** The routing decision between the Queue and Processors is moved from the Queue to the Processors. Processors pull items based on a specified criteria, rather than having items pushed from the Queue. The pull logic is enabled on the Processor’s Flow tab by checking the **Pull** box in the **Input** section of the tab. The values for **Pull Strategy** and **Pull Requirement** are then be specified.



Begin with the basic model from the previous section, named Primer_3-4, and **Save As** Primer_3-5. Basically make a copy of the model so that it can be customized and the original model is retained.



[Section 6-2] Using Lists for decision making

Before proceeding with the model, it may be better to turn off the heat map from the A* algorithm.

- Uncheck the **Show Heat Map** on the Visual tab of the A*Navigator object.

Lists, also called Global Lists, have many uses in modeling. Lists are tools that can create complex flows between objects. There are many types of lists available in *FlexSim* - Fixed Resource List, Item List, Task Sequence List, Task Executer List, and General List. However, in this example, an Item List is used to implement the routing logic specified above.

Information is *pushed to* and *pulled from* lists when events occur. Typically, an event causes an **entry** to be placed on a list and another event causes an entry to be removed from a list. The information about an entry is contained in **fields**. The overall structure of a list is like a table.

As a simple example, consider the simple To-Do list in the figure below. An entry is something that needs to be done and is a row in the list. Entries are put on (pushed to) a list as a result of an event occurring. For example, in the To-Do List, when we think of something or are told to do something, we make an entry onto, or push to, the list. Entries are typically added at the bottom of the list. Entries are removed (pulled) from a list, again when an event occurs. For example, in the To-Do list, an entry is removed, or pulled from, the list when it is completed or deemed no longer necessary to be done.

Fields			
Task	When	Where	Priority
Replace filters	Saturday	Home	Normal
Pay electric bill	Today	Home	High
Grade tests	By Thursday	Office	High
Pickup cleaning	After Tuesday	Out	Normal
Plan summer trip	Anytime	Home	Low

↑
Entries
↓

The columns in the list are the *fields* and represent characteristics of entries, such as in the To-Do List, a brief description of what needs to be done (Task), when it needs to be done (When), where it needs to be done (Where), and how important it is (Priority). Thus, each entry has a *value* associated with each field.

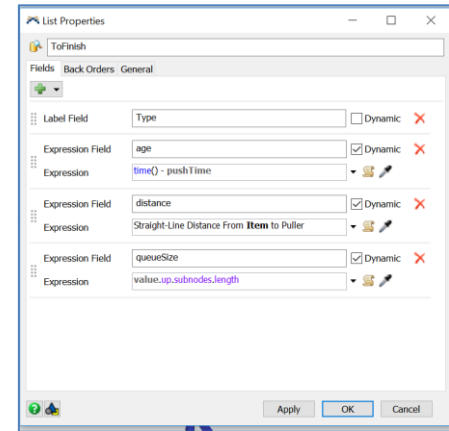
In the case of the primer example, as items enter the Queue, the Queue *pushes* information about the item to a List, i.e., a **list entry** is made. When available, Processors scan all of the list entries and *pull* an item to process based on a specified criteria. The scanning of the list and selecting an item is referred to as a **query**.

The threshold value for the longest time an item should wait is stored in a Global Table called Parameters. This way the threshold value can be easily changed. Actually, simulation experiments with this model will be used to set the threshold value in order to find the right balance of maintaining the SPT rule, but not allowing items to wait too long.

- Set up a new Global Table named **Parameters**. For now, it will contain just one cell; also, assume the threshold value is 30 minutes. Change the Row Heading value from Row1 to Wait Threshold.

- Create an **Item List** from the Global List tool in the Toolbox and name it **ToFinish**, as shown in the figure to the right.

Fields must be defined for the List and how their values are determined for each entry; i.e., what information about the item is put into the list and how that information is determined. Fields may be of different types, e.g. a label value on the item or calculated using an expression that is defined in each field's **Expression** property.



By default, *FlexSim* provides four fields, as shown in the figure to the right and defined below. Fields are either **Dynamic** (updated as affected states change in the simulation) or not (use and maintain the current value).

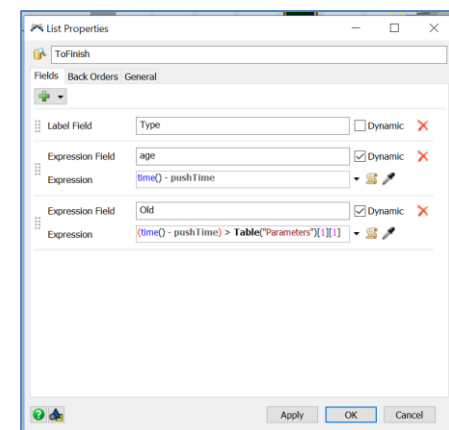
- *Type* is the value of the item's label Type at the time it is placed on the list (not **Dynamic**).
- *age* is how long an entry has been on the list. The value is calculated based on the **Expression** defined as: **time() – pushTime**. This means that the age of an item on this list is the current simulation time, **time()**, minus the simulation time when the entry was put onto the list, **pushTime**. Since the **Dynamic** box is checked, the value is constantly updated while the entry is on the list.
- *distance* is how far (straight-line distance) the item currently is from the object that will pull the entry from the list. The value is calculated based on the **Expression** defined as a drop-down menu option. Since the **Dynamic** box is checked, the value is constantly updated while the entry is on the list.
- *queueSize* is how many items are in the object where the item is currently located. The value is calculated based on the **Expression** defined as a drop-down menu option. Since the **Dynamic** box is checked, the value is constantly updated while the entry is on the list.

In the primer example, only the first two fields are needed. Therefore, the last two can be deleted. Of course, there is no problem leaving them on the List.

- Delete the **distance** and **queueSize** fields by using the red x next to the **Dynamic** property for the field.

In this example, additional information, beyond the default fields, is needed. The routing logic is based on whether an item has waited longer than a threshold value, not just how long it has waited, which is what the **age** field contains.

- Using the + on the List Properties window, just under the Fields tab name, create a new **Expression** field that will indicate if an item is “old,” i.e., has been on the List longer than the threshold value that is specified in the Global Table named Parameters. As shown in the figure to the right:
- Replace the default field name, **fieldName** in the **Expression Field** with the name **Old**.
- Check the **Dynamic** box.
- Replace the **0** in the **Expression** box with the following:
(time() - pushTime) > Table("Parameters")[1][1]
 - The expression to the left of the > (greater than sign) can be copied and pasted from the age expression above. Be sure the expression is within parentheses, (...).



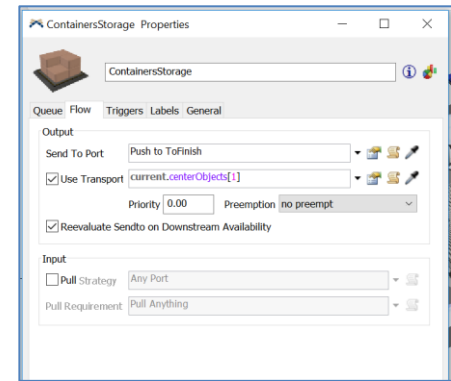
- The context-sensitive editing in *FlexSim* enables the table name to be selected from a list of all Global Tables in the model as the expression is being entered

The “Old” expression checks to see if the amount of time that the entry has been on the List exceeds the threshold value, which is stored in the first row and first column of the Parameters table. If the time on the List exceeds the threshold, then the comparison is true and the field will have a value of 1. If the entry has not be on the List longer than the threshold value, then the comparison is false and the field will have a value of 0.

Now that the List is defined in terms of the information that it contains, the model needs to push information to the List and pull information from the List in order to decide what to process next.

Push an item’s information to the List from the Queue (ContainerStorage), as shown in the figure to the right.

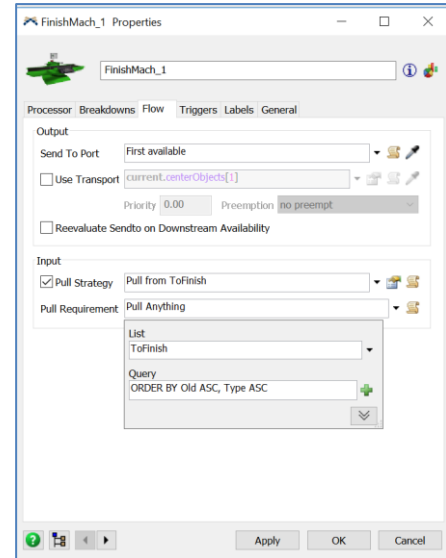
- On the Flow tab, change the Send To Port trigger from **First Available** to **Use List**, and then **Push to Item List**. Make sure the **ToFinish** list is shown in the menu.
- Check the box for **Reevaluate Sendto on Downstream Availability**.
- While in the Queue object, use the outer X icon to remove the **On Entry** trigger **Sort by Expression**. When doing so, a dialogue box will appear asking “Remove this trigger and its logic?”; press OK. The List logic is now controlling the item selection from the Queue so the Queue no longer needs to be sorted when items enter.



The finish machines will use information in the List to decide what to process next. It will:

- query the current List,
- select an entry based on the specified criteria,
- pull the information from the List, and
- pull an item from the Queue based on the information.

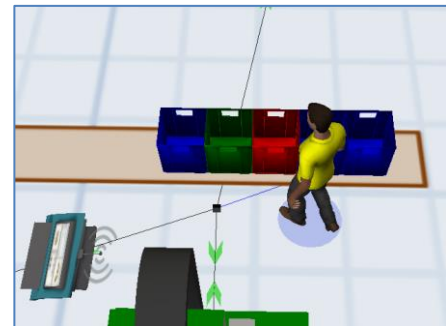
- The above logic, as described below and as shown in the figure to the right, is implemented on the Flow tab of each of the finish machines (Processors).
- Check the **Pull** box on the Input section.
- For **Pull Strategy**, select **Use List**, then **Pull from Item List** from the drop-down menus.
 - Be sure the **List** property value is set to **ToFinish**.
 - Enter two sort queries using the +. The query is to first sort the List based on the Old field, then by the Type field. To do this:
 - Select **Order By (Sort)** from the drop-down menu, then **Old**. Change **ASC** to **DESC**.
 - Again using the +, select **Order By (Sort)** again from the drop-down menu, then **Type**. Leave the default **ASC**.
 - The resulting syntax shown in **Query** should be: **ORDER BY Old DESC, Type ASC**



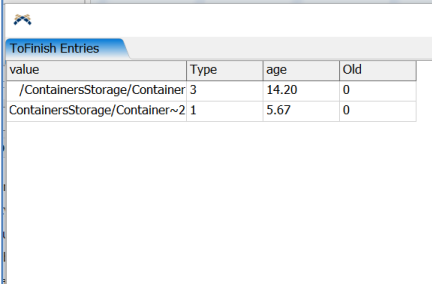
This command will sort the List and place the entries in the specified order. The List is first sorted in descending (DESC) order of the values in the field Old. The values with Old=1 (indicating they have been on the List longer than the threshold) are placed at the top of the list and those with Old=0 are at the bottom of the List. The List is sorted again by the values of item's Type and they are put in ascending (ASC) order of Type. As a result of the query, any entries in the list that are beyond the threshold (Old=1) and a high priority (Type=1) will be at the top of the list.

The finish machine will pull the flowitem from the Queue that is associated with the entry at the top of the List and the entry is subsequently removed from the List.

- Be sure the above pull logic changes are repeated on the other finish machine.
- **Reset** and **Run** the model. Notice, as in the figure to the right, the operator does select the intended item in the queue. The operator is moving to the red container (Type 1) for processing even though there is an item (blue) in front that has waited longer.

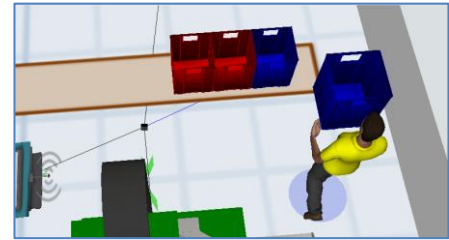


- To view the current contents of the List, double click on the ToFinish List in the Toolbox. On the General tab, select **View Entries...** In the example to the right, based on the criteria, the next one to be processed is the Type 1 even though one other item has been in the queue longer.

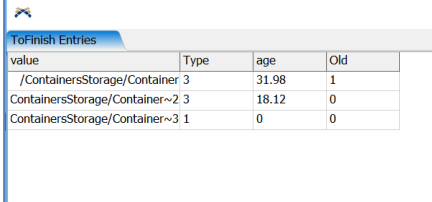


value	Type	age	Old
/ContainersStorage/Container 3	3	14.20	0
ContainersStorage/Container~2 1	1	5.67	0

In the figure to the right, the operator correctly selects the first item in the queue, even though there are higher priority items in the queue. This is because the blue item has been in the queue longer than the 30-minute threshold.

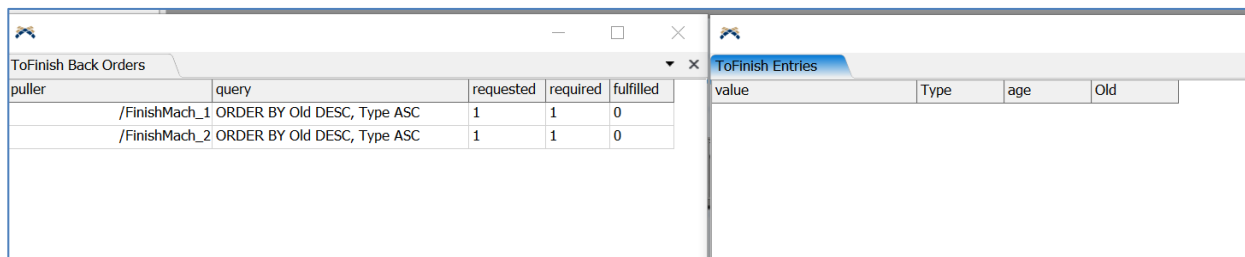


In the example to the right, based on the criteria, the next one to be processed is the Type 3 item even though another item has a higher priority. However, the Type 3 item has waited longer than the specified 30-minute threshold, as indicated by its value for the field Old is 1.



value	Type	age	Old
/ContainersStorage/Container 3	3	31.98	1
ContainersStorage/Container~2 3	3	18.12	0
ContainersStorage/Container~3 1	1	0	0

The **View Back Orders...** option on the General tab of the List Properties displays the resources that are awaiting flowitems. In figure below, the view on the left is for Backorders and the view on the right is for Entries. In the case below, the finish machines are on the Backorders List because they are idle – they want to pull items from the List, but there are none to pull. The Entries list is empty, no items are in the queue awaiting processing.



puller	query	requested	required	fulfilled
/FinishMach_1	ORDER BY Old DESC, Type ASC	1	1	0
/FinishMach_2	ORDER BY Old DESC, Type ASC	1	1	0

value	Type	age	Old
-------	------	-----	-----



If you haven't already done so, save the model. Recall, it is good practice to save often.

There is not a lot of queueing or waiting in the model, at least so far. Therefore, it may be difficult to find good test conditions, especially for the routing rules between the Queue and Processors. One way to test the logic is to temporarily change the Process Times (column 1) values in the Process Times Global Table; e.g., doubling the times. Then the model can be run and the logic tested. Of course, the changed parameter(s) need to be reset to their intended value(s).

4 EXPERIMENTATION IN FLEXSIM

Simulation models are created for analysis. Analysis is based on the output of a simulation model and is used to support decision making and problem solving.

In the models considered so far, output has been limited to a single run. Also, output has also been limited to basic object statistics that are displayed on the object and in the Quick Properties window. While these are good for testing a model's behavior, they are insufficient for analysis.

Analysis must be based on multiple runs of the model, called *replications*.

Analysis also needs to be focused on the *performance measures* that are important to the decision being made or the problem being addressed.

Analyzing simulation models requires a foundation in probability and statistics and includes a variety of methodologies. It is beyond the scope of this primer to discuss simulation analysis - it only introduces the means in *FlexSim* to carry out experiments and obtain output from a simulation model. For more information on simulation analysis, see:



[Chapters 10 and 11] Fundamentals of Output Analysis and Multi-scenario Experimentation and Optimization

Two examples of experimenting with the current simulation model are discussed below.

4.1 Effect of buffer size on performance

The first Experimenter example is used to assess the affect on performance of the size of the containers' buffer (ContainersStorage object) that is located prior to the finishing area. The affect is measured by the number of containers that have to be redirected to another area because there is no space in the buffer for an arriving container.

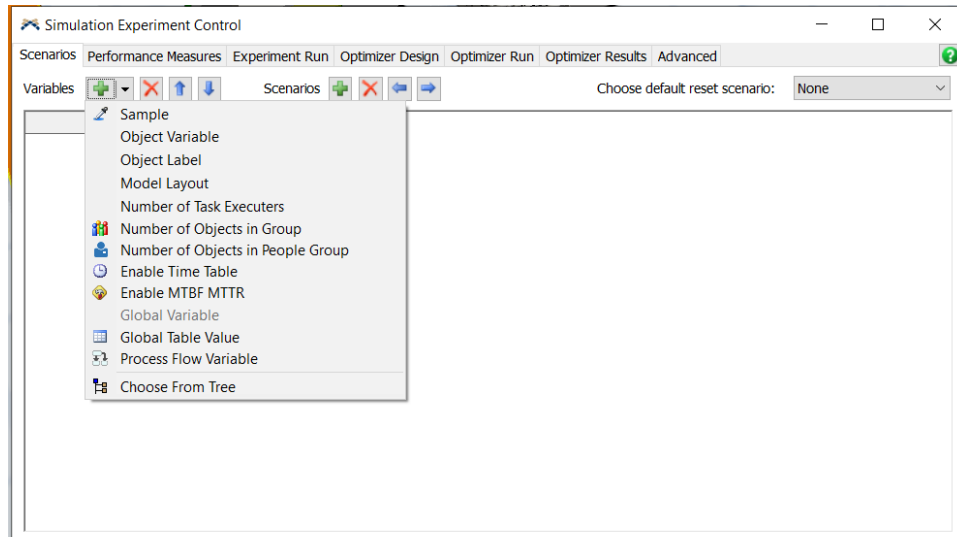


Begin with the basic model from the previous section, named Primer_3-5, and **Save As** Primer_3-6a. Basically make a copy of the model so that it can be customized and the original model is retained. It is denoted as 6a since two experiments are defined below.

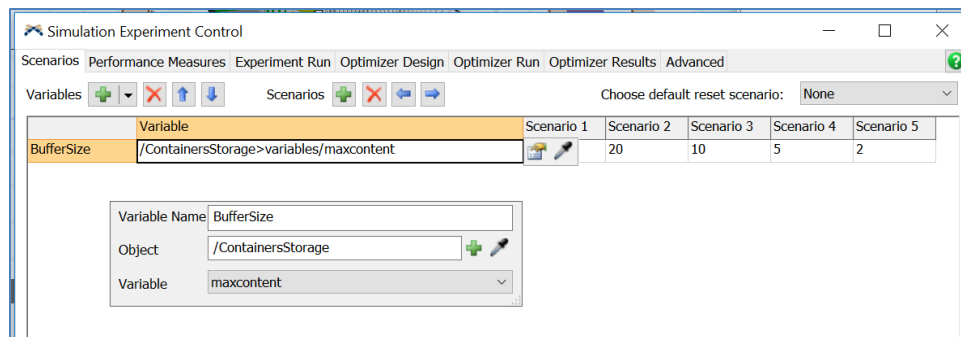
- Launch the **Experimenter** by selecting it from the Statistics drop-down on the Main Menu or from the Statistics section in the Toolbox. This opens the **Simulation Experiment Control** window that contains multiple tabs. Only the first three tabs – Scenarios, Performance Measures, and Experiment Run – are discussed here.

The first tab, Scenarios, is for specifying what alternatives are being considered. These typically represent a combination of values for the decision variables. In the first example, there is one decision variable, the size of the incoming container buffer, i.e., the capacity of the queue ContainersStorage.

- To add a decision variable, click the drop-down menu next to the + in the **Variables** section at the top of the interface, as shown in the figure below.

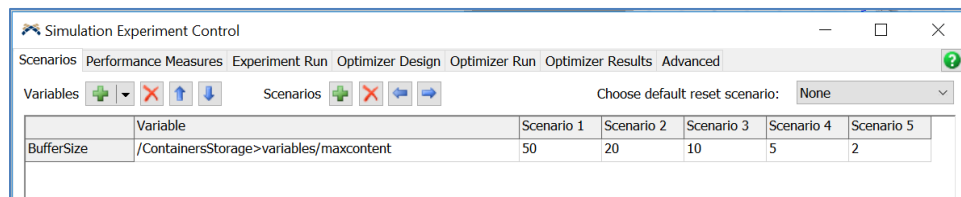


- Select **Object Variable**. And define the properties as follows and as shown in the figure below.
- Change the **Variable Name** from Variable 1 to **BufferSize**.
 - Set the **Object** property by using the + , then select **ContainersStorage** from the Queues category.
 - Use the default value for the **Variable** property, **maxcontent**. This references the *FlexSim* variable that is defined by specifying **Maximum Content** on the Queue tab of the selected Queue object.



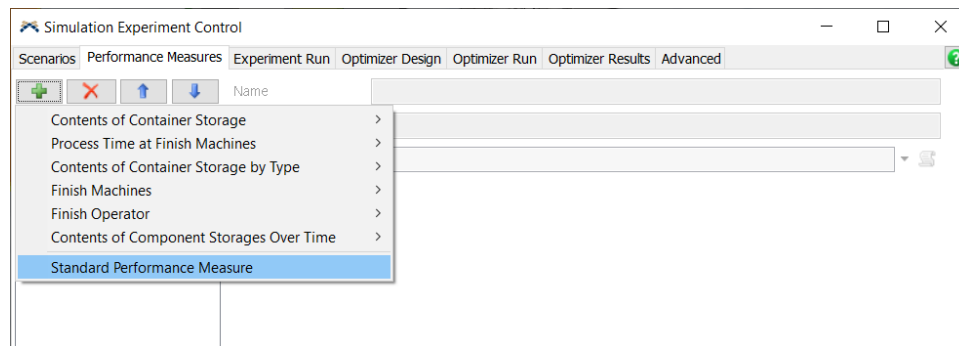
As shown in the figure below, the **Variable** property for each decision variable is a path, defined by *Flexsim*, to reference the object.

- To add and define scenarios, click the + button on the **Scenarios** section at the top of the interface, as shown in the figure below. In this case, create five scenarios and enter the values 50, 20, 10, 5, 2, respectively. Simulations will be run for each of these scenarios in order to evaluate the effect of different buffer sizes. Recall, the currently assumed buffer size is 50.



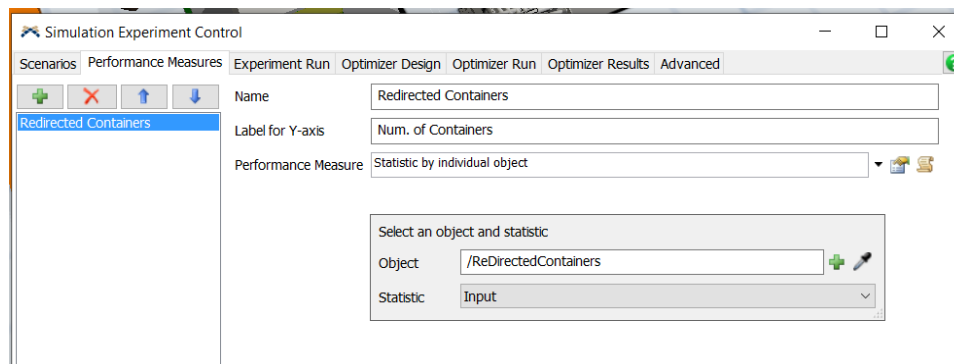
The second tab, Performance Measures, is for specifying how the alternatives are evaluated in order to either (1) understand the effect on the system of changes in the value of the decision variable or (2) to help select the best option. In this example, there is only one performance measure, the number of containers that arrive for which there is no place to store them in the buffer. The measure is the number of items sent to the Sink named ReDirectedContainers.

➤ To add a performance measure, on the Performance Measures tab click the + button and then select from the drop-down menu. As shown in the figure below, select **Standard Performance Measures**, which is at the bottom of the list. Note that the options above it are measures that were created on the Dashboards; thus these measures can be used as Performance Measures in the Experimenter.



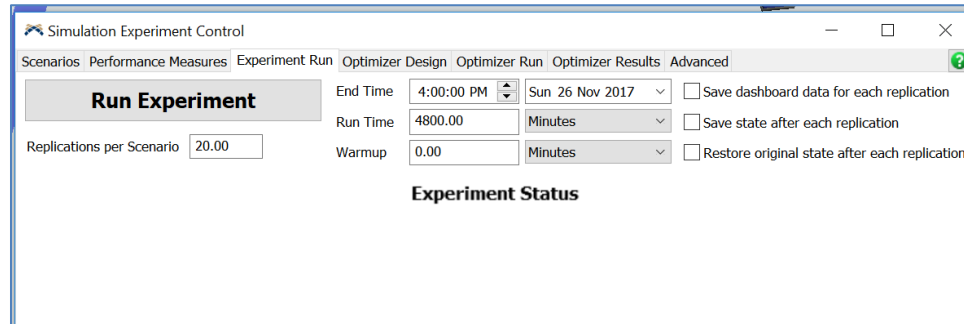
Define the properties for the performance measure as follows and as shown in the figure below.

- Change the **Name** of the measure from PFM1 to **Redirected Containers**.
- Change the **Label for Y-axis** property from the default Value to **Num. of Containers**. On any plot this is the text that describes the units of the measure.
- For **Performance Measure**, select the option **Statistic by individual object** from the drop-down menu. On the resulting interface:
 - Change **Object** from the default Operator1 to the Sink named **ReDirectedContainers** by using the +, then selecting ReDirectedContainers from the Sinks category.
 - Change the property **Statistic** from the default value, Output, to **Input** using the drop-down menu.. Thus, the measure is the total number of items redirected from the buffer that enter the Sink.



The third tab, Experiment Run, is for specifying the run conditions. In this case, just change the following properties, as shown in the figure below.

- **Run Time** to **4800** (minutes).
- **Replications per Scenario** to **20**.

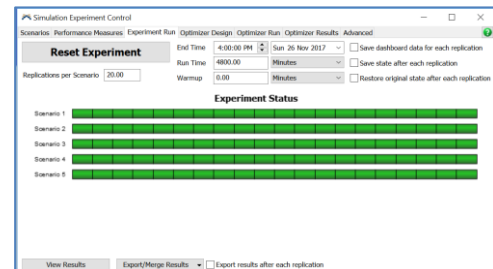


The Experimenter will run each of the five scenarios defined earlier 20 times, with each run being 4800 minutes. Since there are five scenarios and 20 replications of each, 100 80-hour simulations will be performed.

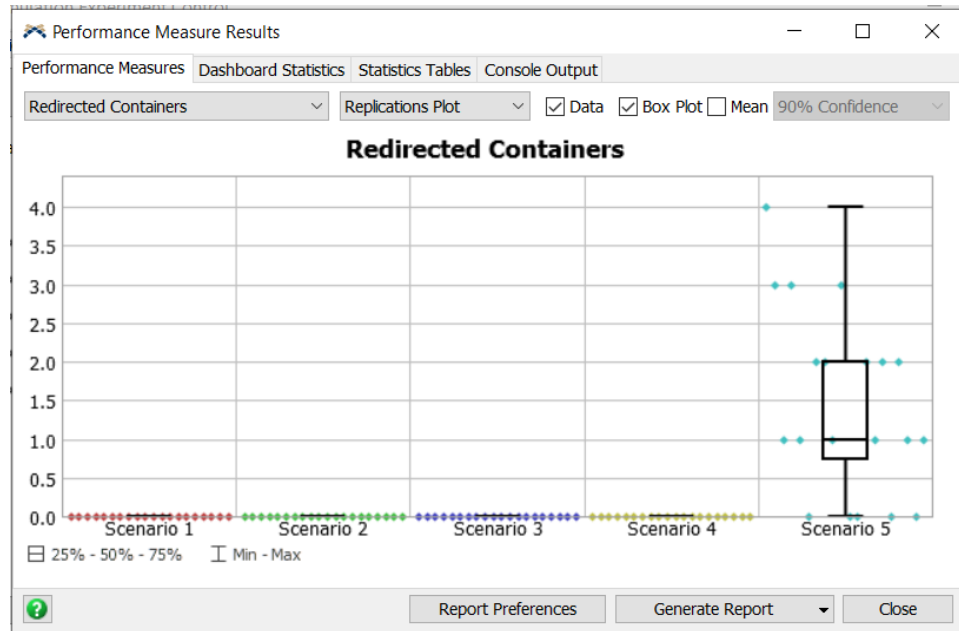
- Press the **Run Experiment** button.

As shown in the figure to the right, as each replication of each scenario is run, its cell in the interface changes from red to green. When all of the cells are green, the results can be viewed.

- To view the result, press the **View Results** button in the lower left portion of the interface.



As shown in the figure below, one output from the Experimenter is a plot of the performance measure value for each scenario and for each replication within the scenario. Note that in this case, items are diverted to the ReDivertedContainers sink only in the fifth scenario, where the Buffer Size is 2. In this scenario, items were diverted due to a full storage area fifteen times in 20 replications. Also in this case, of the 20 replications, no items were diverted five times, one item was diverted six times, two were diverted five times, three were diverted three times, and one item was diverted four times.



Therefore, setting the buffer size to two should not be a problem since only a few of the approximately 240 arriving containers in 80 hours will be diverted. The 240 estimated arrivals is obtained by dividing the simulation run time (80 hours) by the average inter-arrival time (20 minutes). However, in this example, to be a bit conservative, we'll use a buffer size of five,

- Change the **Maximum Content** of the ContainersStorage Queue from 50 to **5**. Recall, this property is on the Queue tab.



If you haven't already done so, save the model. Recall, it is good practice to save often.

4.2 Effect of product mix on performance

The second Experimenter example is used to assess the effect on performance of the mix of products (percentage of Type 1, 2, and 3 containers). The effect is measured by the throughput of containers – the number of containers that leave the packing area per 80 hours (length of a simulation run).



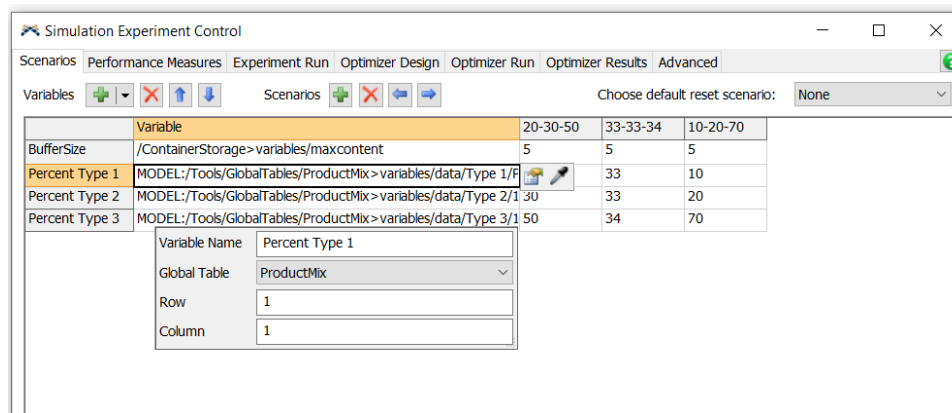
Begin with the basic model from the previous section, named Primer_3-6a, and **Save As** Primer_3-6b. Basically make a copy of the model so that it can be customized and the original model is retained.

Three levels of product mix are considered in this example. The first is the current mix: 20% Type 1, 30% Type 2, and 50% Type 3 (denoted as 20-30-50). The other two mixes are: equal mix, 33-33-34 and more Type 3, 10-20-70. Therefore, three decision variables are added to the Scenarios tab, one for the percent mix of each container type.

Set the properties of the Scenarios tab as shown in the figure below. Recall, the values for the percentage of each type of container are stored in the Global Table ProductMix. Therefore, the Experimenter needs to change the table value for each scenario.

- Add three variables, one for each of the three products. As in the previous example, use the drop-down menu for **Variables** that is just to the right of the + button. However, in each case, select the **Global Table Value** option. For the first type of container:
 - Change the value of **Variable Name** from **Variable 2** to **Percent Type 1**.
 - For the **Global Table** property, select **Product Mix** from the list of tables on the drop-down menu.
 - Leave the default values for **Row** and **Column** at **1** and **1** since the percent value for container type 1 is located in cell (1, 1).

The resulting Variables and **Variable** value for **Percent Type 1** should look as follows.



- Repeat the above steps for container types 2 and 3 as follows.
 - Change **Variable Name** to **Percent Type 2** and **Percent Type 3**, respectively.
 - Set **Row** to **2** and **3**, respectively.
 - The properties **Global Table** and **Column** remain **Product Mix** and **1**, respectively.

As defined above, in this case there are only three scenarios.

- Using the X button to the right of the **Scenarios** section, delete Scenario 4 and Scenario 5.
- Rename the scenarios from **Scenario 1**, **Scenario 2**, **Scenario 3** by changing the column headers to **20-30-50**, **33-33-34**, and **10-20-70**, respectively. This is just text and does not affect the experiment, but does improve the readability of the plots.

Based on the analysis from the previous section, the buffer size is set to 5; and, thus is now the same for all three scenarios. Therefore, this Variable could be deleted; however, it will be retained in case it is changed in further analyses.

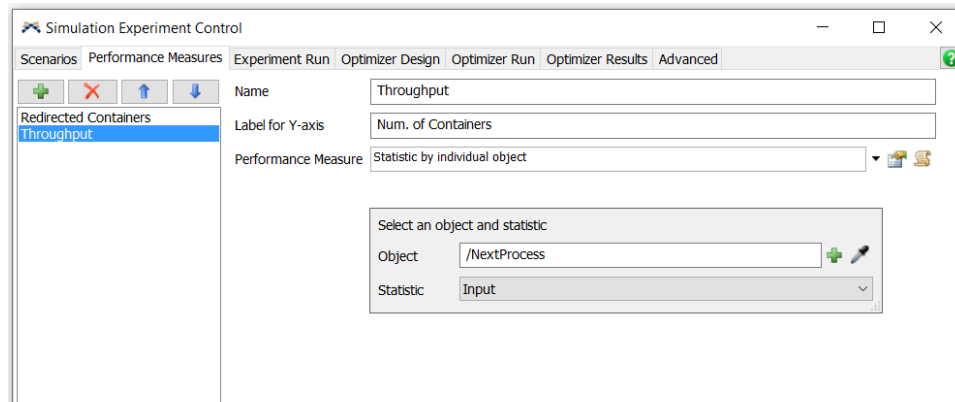
- Change the **Buffer Size** variable to a value of **5** for all scenarios.

The values for each variable are set as follows and as shown in the figure below.

- For the Scenario 1 column, **20-30-50**, the values for the rows **Percent Type 1**, **Percent Type 2**, and **Percent Type 3** should be **20**, **30**, and **50**, respectively.

Defining the properties for the new performance measure, *throughput*, involves basically the same steps as described for the first measure in the previous section. Throughput is defined as the number of items exiting the system, i.e., entering the Sink NextProcess.

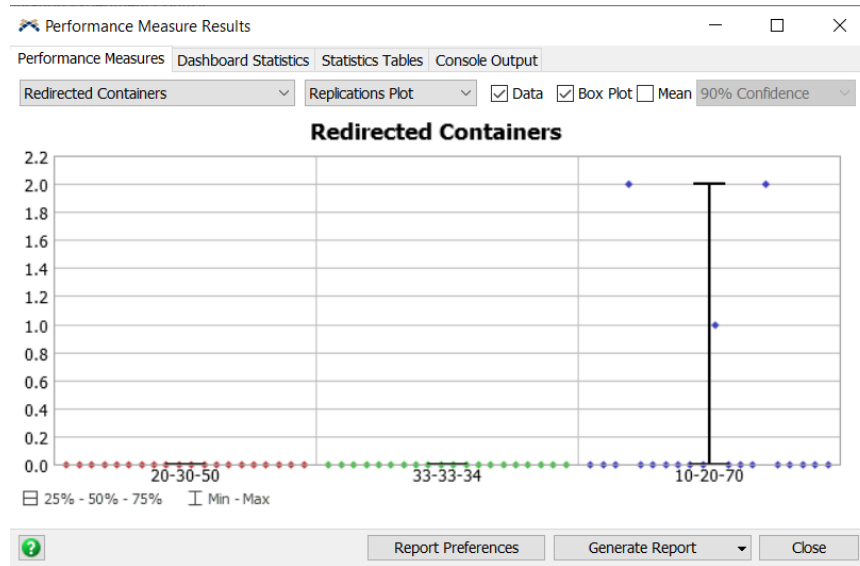
- To add the performance measure, on the Performance Measures tab of the Experimenter, click the + button and then select **Standard Performance Measures** from the drop-down menu.
- Change the **Name** of the measure to **Throughput**.
- Change the **Label for Y-axis** property from the default Value to **Num. of Containers**. On any plot this is the text that describes the units of the measure.
- For **Performance Measure**, select the option **Statistic by individual object** from the drop-down menu. On the resulting interface:
 - Change **Object** from the default Operator1 to the Sink named **NextProcess** by using the +, then selecting NextProcess from the Sinks category.
 - Change the property **Statistic** from the default value, Output, to Input using the drop-down menu.. Thus, the measure is the total number of items that leave the packing area and enter the Sink.



The third tab, Experiment Run, is not changed from the previous experiment, each scenario is replicated 20 times, with each replication being 4800 minutes.

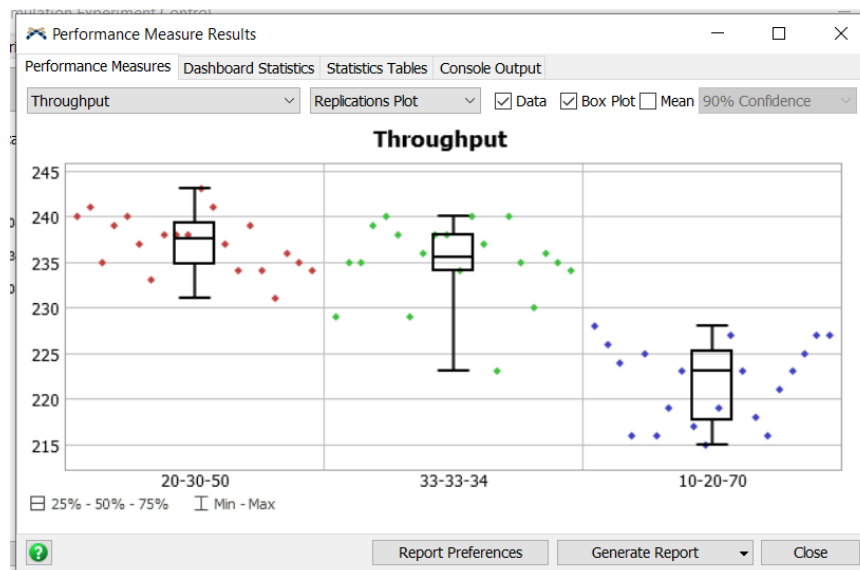
- On the Experiment Run tab, press **Reset Experiment**, then **Run Experiment**, and when all of the cells are green, view the results by pressing the **View Results** button in the lower left portion of the interface.

As shown in the figure below, the product mix with the most Type 3 containers impacts the number of items diverted to the ReDivertedContainers sink. A buffer size of five is used for all cases. In the third scenario, items were diverted in three of the 20 replications, with the most diversions over the 80-hour simulation period being two.



However, throughput is the main concern.

- To see the impact on throughput, click on the drop-down menu in the upper left corner of the interface to change the plot from **Redirected Containers** to **Throughput**. As shown in the figure below, there is not much difference in average throughput between the first two scenarios. However, the third scenario, with the largest percentage of Type 3 containers, results in about a 6% reduction in average throughput.



If you haven't already done so, save the model. Recall, it is good practice to save often.

5 SUMMARY OF THE PRIMER FOR FLEXSIM3D

Even though the current model, Primer_3-6b as shown in the figure below, is not very large, it demonstrates many of the key features of *FlexSim* simulation software. The model was built by customizing basic 3D objects in *FlexSim*. and utilizing a number of model-building tools (shown on the left side of the figure below). The model:

- is built natively in 3D.
- is constructed on an imported layout and thus is built to scale in terms of sizes and distances.
- uses standard pre-built *FlexSim* objects that provide basic functionality, e.g., Processors, Operators, Conveyors, and Combiners. These objects:
 - represent fixed and mobile resources.
 - are customized by specifying properties via drop-down menu options and tools.
- considers multiple types of items that have different characteristics; these characteristics affect various aspects of operations, such as processing time, batch size, routing, etc. Labels on objects and items are used to represent the characteristics.
- uses fixed resources to include various types of activities: processors for planned delays, queues for unplanned delays, conveyors for transport, combiners for grouping, separators for ungrouping, etc.
- uses mobile or shared resources for transportation and tasks (unbatching items).
- uses two different methods for controlling the path of mobile resources, such as Operators: path network and the A* algorithm.
- includes planned downtime (e.g. operator breaks and quality checks).
- includes unplanned downtime (e.g. machine failure).
- includes randomness in the time between arrivals of the base item (container) and in the type of arrival.
- includes scheduled arrivals (of components) – time and batch size.
- utilizes both simple and complex routing rules; uses Lists to incorporate complex rules.
- includes imported custom graphics to make the model more realistic.
- utilizes tables for effective data management.
- uses color to facilitate model validation – different colored product and component types and different colored states (down, on break, etc.).
- uses charts and graphs on dashboards to illustrate the dynamic behavior of the system.
- is used for analysis and deciding among alternatives via *FlexSim*'s Experimenter.

